# Policy Optimization: From CPI to LLMs

Conservative updates, trust regions, advantage estimation, clipping, and preference optimization — companion notes to the seminar decks

Yakov

November 2026

## How to read these notes

These notes follow the two seminar decks and keep the same storyline. They add explanations and derivations; later we decide what goes on the slides.

**Warm (copper)**

- **Part I:** approximate **value-function** methods.
- **Later:** lane **A** — *how far may I step?* (mixture step, trust region, clip).
- **GAE / LLMs:** the value side (critic, reward model).

**Cool (blue)**

- **Part I: policy-gradient** methods.
- **Later:** lane **B** — *where do I learn?* (restart distribution $\mu$).
- **LLMs:** the policy being trained.

### Box types

**Definition: name**

a definition

**Theorem**

a result from a paper

**Key idea**

the take-away

**Limitation**

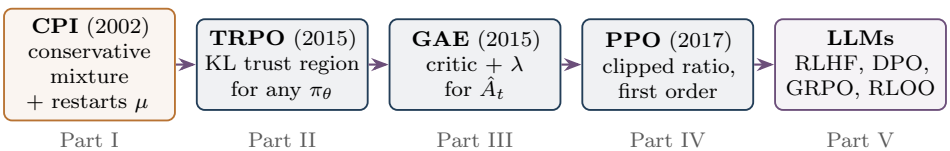
what it does not do

**Algorithm: name**

the steps

> a remark or intuition

### The storyline at a glance




> **Two normalizations.** CPI (Part I) uses **normalized** values: $V_\pi = (1-\gamma)\mathbb{E}[\sum_t \gamma^t r_t] \in [0, R]$, and $d_{\pi,\mu}$ sums to 1. TRPO, GAE and PPO use **unnormalized** returns $\eta = \mathbb{E}[\sum_t \gamma^t r_t]$ and visitations $\rho_\pi$. Constants in the bounds differ by factors of $(1-\gamma)$.

**Sources** (local copies in `seminar/RL_algos/papers/`)

| Key | Paper | Part |
|---|---|---|
| K&L 2002 | Kakade & Langford, *Approximately Optimal Approximate Reinforcement Learning* (ICML) | I |
| TRPO | Schulman et al., *Trust Region Policy Optimization* (ICML 2015) | II |
| GAE | Schulman et al., *High-Dimensional Continuous Control Using Generalized Advantage Estimation* (ICLR 2016) | III |
| PPO | Schulman et al., *Proximal Policy Optimization Algorithms* (arXiv 2017) | IV |
| Human prefs | Christiano et al., *Deep RL from Human Preferences* (NeurIPS 2017) | V |
| InstructGPT | Ouyang et al., *Training Language Models to Follow Instructions with Human Feedback* (NeurIPS 2022) | V |
| DPO | Rafailov et al., *Direct Preference Optimization* (NeurIPS 2023) | V |
| DeepSeekMath | Shao et al., *DeepSeekMath* — introduces GRPO (arXiv 2024) | V |
| RLOO | Ahmadian et al., *Back to Basics: REINFORCE-Style Optimization ...* (ACL 2024) | V |

# Contents


**How to read these notes** **1**

**I Conservative Policy Iteration** **7**

**1 Setting and notation** **7**

1.1 Assumptions . . . 7

1.2 Why the start distribution $D$ matters . . . 7

1.3 Notation: $V$, $Q$, $A$, $d$, $\eta$ . . . 8

**2 Three questions for any approximate method** **9**

**3 Gap 1: approximate value functions** **9**

3.1 Reminder: approximate policy iteration . . . 9

3.2 How is $\tilde{Q}$ estimated? From TD(0) to Monte Carlo . . . 10

3.3 The bound: no improvement guarantee . . . 10

3.4 The bound is tight: a two-state example . . . 11

**4 Gap 2: policy gradients** **11**

4.1 Reminder: policy gradient methods . . . 11

4.2 Policy gradient is stochastic gradient ascent . . . 12

4.3 What SGD theory guarantees . . . 12

4.4 The signal-to-noise reality . . . 12

4.5 Example 1: exploration . . . 13

4.6 Example 2: plateaus . . . 13

4.7 Gap 2 as covariate shift . . . 13

**5 The two fixes** **14**

5.1 Roadmap: from the two fixes to the three answers . . . 15

**6 The performance difference lemma** **15**

**7 The policy advantage** **16**

7.1 The exact change vs. the policy advantage . . . 16

**8 How big a step? The improvement bound** **17**

8.1 Choosing the step: Corollary 4.2 . . . 17

**9 The CPI algorithm** **18**

9.1 Verifying an update: estimating $\mathbb{A}$ from $\mu$-restarts . . . 18

**10 How good is the policy found?** **19**

**11 Summary of Part I** **19**

11.1 The two lanes to follow . . . 20

**II Trust Region Policy Optimization** **21**

**12 Setting: what changes from CPI** **21**

**13 The same identity, a local surrogate** **21**

**14 From mixtures to any stochastic policy: Theorem 1** **22**

**15 From TV to KL: a minorize–maximize algorithm** **23**

**16 Five practical simplifications** **23**

16.1 1. From penalty to constraint: the trust region . . . 23

16.2 2. From max KL to mean KL . . . 24

16.3 Tool: importance sampling . . . 24

16.4 3. Turning sums into samples . . . 25

16.5 Two sampling schemes . . . 25

16.6 4. Linear objective, quadratic constraint: the search direction . . . 25

16.7 5. Computing the step: conjugate gradient + line search . . . 26

**17 The TRPO algorithm** **27**

17.1 Recap: from theory to practice . . . 27

**18 What TRPO does not fix: where the data come from** **27**

**19 Results** **28**

19.1 Simulated robots (MuJoCo) . . . 28
19.2 Atari from raw pixels . . . 28

**III Generalized Advantage Estimation** **29**

**20 Where TRPO leaves us** **29**

**21 Setting** **29**
21.1 What GAE fixes — and what it does not . . . 30

**22 Which weight multiplies the score function?** **30**

**23 GAE: TD($\lambda$) for advantages** **30**

**24 Two knobs, two kinds of bias** **31**

**25 Actor–critic: policy and value networks** **32**

**26 Results: choosing $\gamma$ and $\lambda$** **33**

**IV Proximal Policy Optimization** **34**

**27 Setting and motivation** **34**
27.1 The goal: many SGD steps per batch . . . 34

**28 Back to the CPI surrogate** **35**

**29 The clipped surrogate objective** **35**
29.1 What clipping replaces . . . 36

**30 PPO in practice** **36**

**31 TRPO vs. PPO** **37**

**32 Results** **37**
32.1 Clipping vs. KL penalty: the ablation . . . 37
32.2 PPO vs. other algorithms . . . 38

**33 The whole story: CPI → TRPO → GAE → PPO** **38**
33.1 One line per paper . . . 39

**V RL for Language Models** **40**

**34 Text generation as an RL problem** **40**
34.1 RLHF with PPO: what can we remove? . . . 40

**35 Learning the reward from human preferences** **41**
35.1 Three processes, running asynchronously . . . 41
35.2 Fitting the reward: the Bradley–Terry model . . . 41

**36 InstructGPT: RLHF at LLM scale** **42**
36.1 The three-step recipe . . . 42
36.2 The KL-regularized objective (PPO-ptx) . . . 43

**37 DPO: the RLHF objective as a classification loss** **43**
37.1 From the RLHF objective to a loss on the policy . . . 43
37.2 What the DPO update does . . . 44
37.3 What we removed, and what it costs . . . 44

**38 GRPO: PPO without the critic** **45**
38.1 Group-relative advantages . . . 45
38.2 PPO vs. GRPO . . . 46

**39 RLOO: back to REINFORCE** **46**
39.1 Which parts of PPO does RLHF need? . . . 47
39.2 The leave-one-out baseline . . . 47

**40 Summary of Part V** **48**
40.1 What did we remove? . . . 48
40.2 The thread: from CPI to LLMs . . . 48

**Appendix** **49**

**A Proof: the policy advantage is the derivative at $\alpha = 0$** **49**

**B Comparing with Bertsekas & Tsitsiklis** **49**

**C From $\mathbb{E}_{s\sim d}$ to a sum over time** **51**

**D The Gibbs (softmax) policy** **52**

**E Why the log-derivative trick is unbiased** **53**

**F CPI through the SGD lens** **53**

**G Building $G_\varepsilon$ by regression on advantages** **54**

**H Stopping with probability $1-\gamma$ samples $d_{\pi,\mu}$** **54**

**I Hoeffding's inequality and the estimate of $\mathbb{A}$** **55**

**J Importance sampling in CPI, and why uniform actions** **55**

**K TRPO with continuous states and actions** **56**

**L One problem, several known algorithms** **56**

# Part I

# Conservative Policy Iteration

**The question of Part I:** approximate methods give no guarantee that an update helps. CPI adds two fixes — a **conservative (mixture) step** and a **restart distribution** $\mu$ — and gets answers to three questions: what improves, how to verify it, and how good the result is.

## 1 Setting and notation

**Approximately Optimal Approximate Reinforcement Learning**
Kakade & Langford · ICML 2002

**Introduces:** Conservative Policy Iteration (CPI)
Local copy: `approximately_optimal_approximate_RL.md`

### 1.1 Assumptions

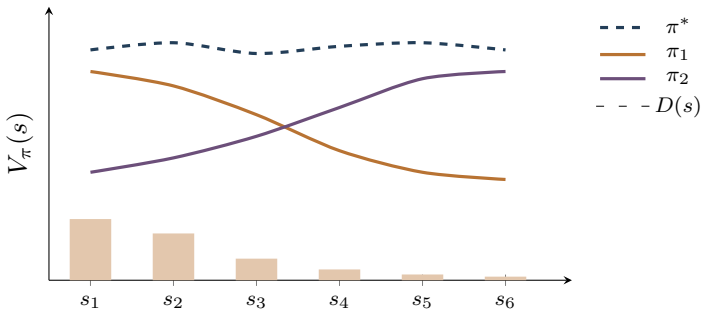
A finite MDP $(S, D, A, \mathcal{R}, P)$:

1. **Finite** state and action spaces: $|S|, |A| < \infty$. TRPO later allows continuous spaces
2. **Bounded rewards**: $\mathcal{R} : S \times A \to [0, R]$.
3. **Discounted**: $\gamma \in [0, 1)$.
4. **Unknown dynamics**: $P(s'; s, a)$ is not given; we can only **sample** transitions (model-free).
5. **Start-state distribution** $D$. The goal is

$$\eta_D(\pi) = \mathbb{E}_{s_0 \sim D}\big[V_\pi(s_0)\big].$$

Finiteness does not help by itself: $P$ is unknown, so $V_\pi$ can only be *estimated* from rollouts, never computed by dynamic programming. The paper also assumes an approximate greedy policy chooser (a black box); it enters with the algorithm (§9).

### 1.2 Why the start distribution $D$ matters




**Exact**

$\pi^*$ is on top at **every** state $\Rightarrow$ $D$ is irrelevant.

**Approximate**

Curves **cross**: a restricted class $\{\pi_\theta\}$ and limited data.

**Rank by $D$**

$\eta_D(\pi) = \sum_s D(s) V_\pi(s)$. Here $\pi_1$ wins.

**Key idea**

$D$ comes from the **application** (robot start poses, game openings, user prompts). It is fixed by the problem, like $\mathcal{R}$ and $P$, and it **defines the goal**. The restart distribution $\mu$ (later) is different: we choose it, and it only shapes the *learning signal*.

Two reasons why the curves cross:

1. **Representation** — $\pi^*$ may not be in the class. Persists with unlimited data.
2. **Estimation** — even in the tabular case CPI stops with $\mathrm{OPT}(\mathbb{A}_{\pi,\mu}) < 2\varepsilon$ (Thm. 4.4); the leftover error is uneven across states (larger where samples are rare). A finite-data effect.

**Source:** K&L Sec. 2–3. "We only consider the case where the goal of the agent is to maximize the $\gamma$ discounted average reward from the starting state distribution $D$."

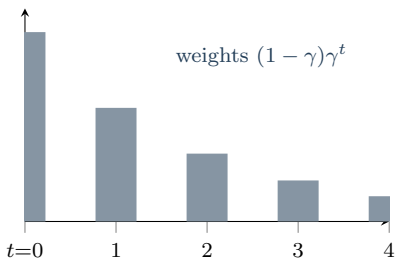
## 1.3 Notation: $V$, $Q$, $A$, $d$, $\eta$

Five quantities, each built on the previous one.

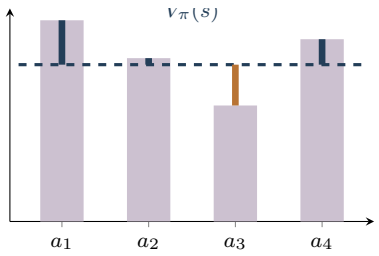
**Definition: value, action value, advantage (normalized)**

$$V_\pi(s) = (1-\gamma)\,\mathbb{E}\Big[\textstyle\sum_t \gamma^t \mathcal{R}(s_t,a_t)\,\Big|\,\pi, s_0{=}s\Big]$$
$$Q_\pi(s,a) = (1-\gamma)\mathcal{R}(s,a) + \gamma\,\mathbb{E}_{s'}\big[V_\pi(s')\big], \qquad V_\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\big[Q_\pi(s,a)\big]$$
$$A_\pi(s,a) = Q_\pi(s,a) - V_\pi(s)$$

Value: a weighted average of rewards




Advantage: the gap to $V_\pi(s)$




- **Value.** The weights $(1-\gamma)\gamma^t$ sum to 1, so $V_\pi$ is an *average* of rewards: $V \in [0, R]$.
- **Action value.** Take $a$ first, then follow $\pi$. $V_\pi$ is the $\pi$-weighted average of the bars.
- **Advantage.** Gaps to the $V$ line. Weighted by $\pi$ they cancel: $\mathbb{E}_{a\sim\pi}[A_\pi(s,a)] = 0$. A different $\pi'$ that favors the **positive** gaps gets $\mathbb{E}_{a\sim\pi'}[A_\pi(s,a)] > 0$ — the seed of the *policy advantage* (§7).

**Definition: discounted visitation and objective**

$$d_{\pi,D}(s) = (1-\gamma)\textstyle\sum_t \gamma^t \Pr(s_t = s;\,\pi, D), \qquad \eta_D(\pi) = \mathbb{E}_{s\sim D}\big[V_\pi(s)\big] = \mathbb{E}_{s\sim d_{\pi,D},\,a\sim\pi}\big[\mathcal{R}(s,a)\big]$$

**$d_{\pi,D}$: the whole trajectory**

Where $\pi$ spends its time when started from $s_0 \sim D$; early steps weigh more.

**$D$: only $s_0$**

The distribution of the first state. Easy to confuse with $d_{\pi,D}$.

- Unnormalized visitation: $\rho_\pi = (I - \gamma P_\pi^\top)^{-1} D$ — linear in $D$, nonlinear in $\pi$.
- Training is on-policy: $\pi$ shapes the visitation used for the next update — a **feedback loop**, not a fixed reweighting.
- TRPO drops the $(1-\gamma)$ factor; this changes the constants in its bounds.

# 2 Three questions for any approximate method

**The three questions (K&L Sec. 3)**

**Q1** Is there a **performance measure** that is guaranteed to **improve at every step**?
**Q2** How hard is it to **verify** that a given update improves this measure?
**Q3** What **performance level** is reached after a **reasonable number** of updates?

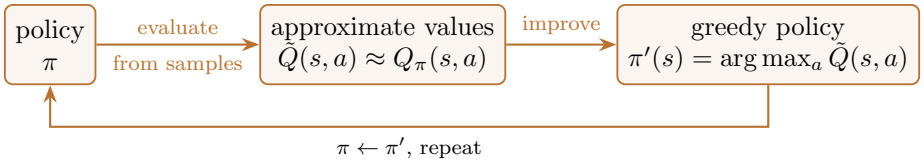
Not "what is the asymptotic policy?" — only policies reachable in reasonable time.

| | **Q1** improve | **Q2** verify | **Q3** performance |
|---|---|---|---|
| **Approximate value functions** | weak | weak | weak |
| **Policy gradients** | **good** | poor | poor |
| **CPI** | ✓ | ✓ | ✓ (up to a mismatch factor) |

The next two sections show why the first two rows look like this (**Gap 1** and **Gap 2**).

# 3 Gap 1: approximate value functions

## 3.1 Reminder: approximate policy iteration



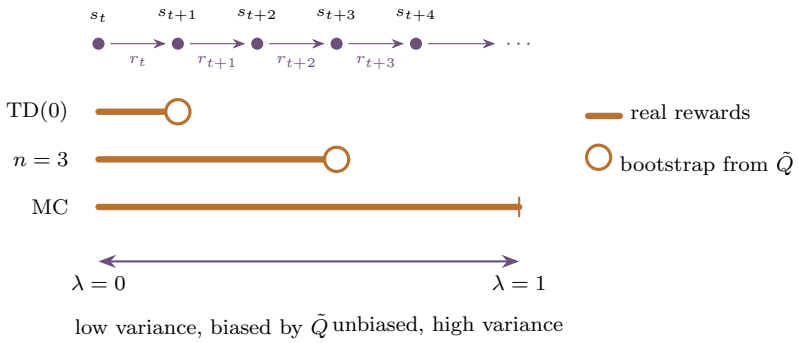

Exact policy iteration, with $Q_\pi$ replaced by an **estimate** $\tilde{Q}$.

**Example: Q-learning with a function approximator $\tilde{Q}_w$**

$$w \leftarrow w + \alpha \left(y_t - \tilde{Q}_w(s_t, a_t)\right) \nabla_w \tilde{Q}_w(s_t, a_t), \qquad y_t = (1-\gamma)\, r_t + \gamma \max_{a'} \tilde{Q}_w(s_{t+1}, a')$$

- The max is the greedy step: evaluate and improve happen **together**.
- Strictly, this is approximate *value* iteration; with a neural network it is DQN.
- The target $y_t$ is one choice of many (next subsection).

### 3.2 How is $\tilde{Q}$ estimated? From TD(0) to Monte Carlo




| Target | Formula | Trade-off |
|---|---|---|
| TD(0) | $y_t^{(1)} = (1-\gamma)\, r_t + \gamma\, \tilde{Q}(s_{t+1}, a_{t+1})$ | one reward, then trust $\tilde{Q}$ |
| $n$-step | $y_t^{(n)} = (1-\gamma) \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n\, \tilde{Q}(s_{t+n}, a_{t+n})$ | in between |
| Monte Carlo | $y_t^{(\infty)} = (1-\gamma) \sum_{k\geq 0} \gamma^k r_{t+k}$ | only real rewards; wait for the end |
| TD($\lambda$) | $y_t^{\lambda} = (1-\lambda) \sum_{n\geq 1} \lambda^{n-1}\, y_t^{(n)}$ | average over all $n$ |

The weights $(1-\lambda)\lambda^{n-1}$ sum to 1: $\lambda = 0$ gives TD(0), $\lambda \to 1$ gives Monte Carlo. The **same $\lambda$ trade-off returns for advantages** in GAE (Part III). Background from Sutton & Barto, Ch. 6, 7, 12; not in K&L.

### 3.3 The bound: no improvement guarantee

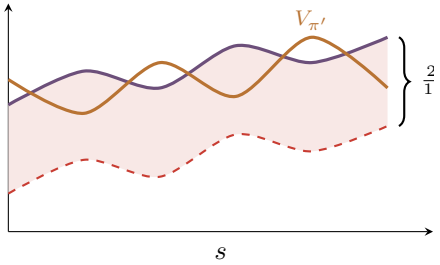
Measure the approximation error **everywhere** ($\ell_\infty$):

$$\varepsilon = \max_s \left|\tilde{V}(s) - V_\pi(s)\right|.$$

**Approximate greedy step (Bertsekas & Tsitsiklis; K&L Eq. 3.1)**

If $\pi'$ is greedy with respect to $\tilde{V}$, then

$$V_{\pi'}(s) \;\geq\; V_\pi(s) - \frac{2\gamma\varepsilon}{1-\gamma} \qquad \forall s.$$

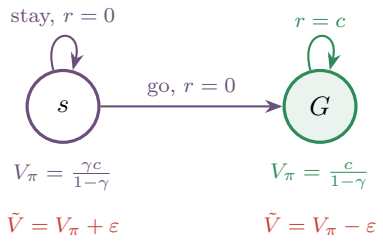



- $V_{\pi'}$ may sit anywhere above the dashed line — **also below** $V_\pi$.
- Only "won't get **much worse**"; never "gets better".
- The $\ell_\infty$ requirement is strong: one badly estimated state is enough to make the bound loose.

**Limitation**

Weak answers to all three questions: no measure is guaranteed to improve (Q1), so verifying an update (Q2) has nothing to check against, and nothing bounds the performance after a number of updates (Q3).

### 3.4 The bound is tight: a two-state example




$\pi$ = go. Set $c = 2\varepsilon$. Unnormalized values.

1. $\pi$ goes to $G$ and collects $c$ forever.
2. $\tilde{V}$ is off by $\varepsilon$: too **high** at $s$, too **low** at $G$.
3. Greedy w.r.t. $\tilde{V}$ at $s$ compares

$$\underbrace{\gamma\big(V_\pi(s) + \varepsilon\big)}_{\text{stay}} \;=\; \underbrace{\gamma\big(V_\pi(G) - \varepsilon\big)}_{\text{go}}$$

a tie, since $V_\pi(G) - V_\pi(s) = c = 2\varepsilon$. So $\pi'$ may stay forever: $V_{\pi'}(s) = 0$.

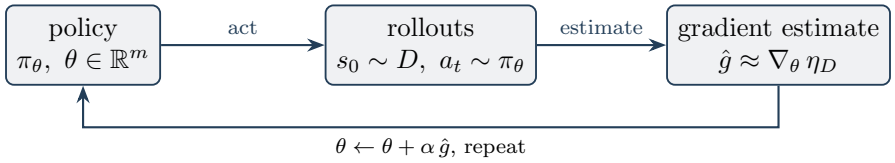
**The loss equals the bound**

At $s$: $V_\pi(s) - V_{\pi'}(s) = \frac{\gamma c}{1-\gamma} = \frac{2\gamma\varepsilon}{1-\gamma}$. At $G$: one action only, so the loss is 0.
Two errors of size $\varepsilon$, in opposite directions, make the greedy policy stop moving; the $\frac{1}{1-\gamma}$ sums the lost reward over the horizon.

Our own minimal example. With $c$ slightly below $2\varepsilon$ staying is strictly preferred and the loss gets arbitrarily close to the bound. B&T (*Neuro-Dynamic Programming*, Ex. 6.4) show tightness of the related API bound $\limsup_k \|V_{\pi_k} - V^*\|_\infty \le \frac{2\gamma\varepsilon}{(1-\gamma)^2}$; the one-step form is due to Singh & Yee (1994).

## 4 Gap 2: policy gradients

### 4.1 Reminder: policy gradient methods




**Policy gradient theorem (Sutton et al.; normalized)**

$$\nabla_\theta \eta_D = \frac{1}{1-\gamma} \sum_{s,a} d_{\pi,D}(s)\, \nabla_\theta \pi(a;s)\, Q_\pi(s,a)$$

Small steps uphill on $\eta_D$ *itself* $\Rightarrow$ a **good answer to Q1**.

**Example: REINFORCE with a Gibbs (softmax) policy**

$$\pi_\theta(a;s) = \frac{e^{\theta_{sa}}}{\sum_b e^{\theta_{sb}}}, \qquad \hat{g} \;\propto\; \sum_t \nabla_\theta \log \pi_\theta(a_t;s_t)\, \hat{Q}_t$$

$\hat{Q}_t$: the return observed after step $t$. The Gibbs class is the one of Example 2 below. Details: App. D.

## 4.2 Policy gradient is stochastic gradient ascent

| | Stochastic gradient descent | Policy gradient |
|---|---|---|
| objective | $\min_w J(w) = \mathbb{E}_X[f(w, X)]$ | $\max_\theta \eta_D(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$ |
| looking for | $\nabla J(w) = 0$ | $\nabla_\theta \eta_D(\theta) = 0$ |
| one sample | $x_k \sim X$ | rollout $\tau_k$: $s_0 \sim D$, $a_t \sim \pi_{\theta_k}$ |
| update | $w_{k+1} = w_k - a_k \nabla_w f(w_k, x_k)$ | $\theta_{k+1} = \theta_k + a_k \hat{g}_k$ |
| noise | $\eta_k = \nabla_w f(w_k, x_k) - \nabla J(w_k)$ | $\eta_k = \hat{g}_k - \nabla_\theta \eta_D(\theta_k)$ |

**The twist.** In SGD the distribution of $X$ does not depend on $w$. Here the trajectories come from $\pi_\theta$ itself, so $\nabla\eta_D \neq \mathbb{E}[\nabla R]$. The log-derivative trick (REINFORCE) still gives an **unbiased** $\hat{g}_k$ — App. E. Notation clash: $\eta_k$ (noise) is not $\eta_D$ (performance).

## 4.3 What SGD theory guarantees

**Convergence conditions** (Robbins & Monro, 1951) — each has a role:

| Condition | Formula | Role |
|---|---|---|
| 1. Curvature | $0 < c_1 \leq \nabla_w^2 f(w, X) \leq c_2$ | one minimizer; bounds on the rate |
| 2. Unbiased noise | $\mathbb{E}[\eta_k \mid \mathcal{H}_k] = 0$ | steps point the right way on average |
| 3. Finite variance | $\mathbb{E}[\eta_k^2 \mid \mathcal{H}_k] < \infty$ | with 4, the summed noise stays finite |
| 4. Step sizes | $\sum_k a_k = \infty,\ \sum_k a_k^2 < \infty$ | reach any distance; e.g. $a_k = c/k$ |

$\mathcal{H}_k$: the history so far, $w_0, x_0, \ldots, x_{k-1}$. Under 1–4: $w_k \to w^*$ almost surely.

**Sample complexity** — samples needed to be $\epsilon$-close with probability $\geq 1 - \delta$:

$$N(\epsilon, \delta) = \min\left\{N \in \mathbb{N} \;\middle|\; \mathbb{P}\big(\|w_k - w^*\| \leq \epsilon\big) \geq 1 - \delta \;\text{ for all } k \geq N\right\}$$

| Setting | Guarantee |
|---|---|
| Convex, smooth | global optimum: $N(\epsilon, \delta) = O\big(\sigma^2/(c_1^2\epsilon^2\delta)\big)$ — Nemirovski et al. 2009 + Markov |
| Non-convex, smooth | stationary point only: $\mathbb{E}\|\nabla J(w_R)\| \leq \epsilon$ after $K = O(\sigma^2/\epsilon^4)$, $w_R$ a random iterate — Ghadimi & Lan 2013 |
| Non-convex, non-smooth | weaker still: near-stationary points, special classes — **policy gradients with neural nets live here** |

**What carries over**

The convex case shows **what drives** $N$: the noise $\sigma^2$ and the curvature $c_1$. More noise means more samples; flat regions are slow.

## 4.4 The signal-to-noise reality

1. Finite variance is enough for convergence *in the limit.*

2. In practice $N$ depends on the **signal-to-noise ratio**. Averaging $n$ samples leaves noise $\sigma/\sqrt{n}$; just to get the *direction* right:

$$n \gtrsim \sigma^2/\|\nabla J\|^2.$$

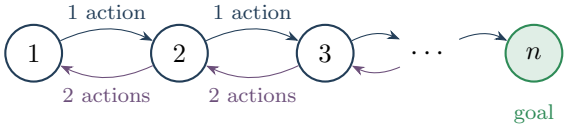
3. **Link to curvature.** Strong convexity gives $\|\nabla J(w)\| \geq c_1\|w - w^*\|$, so at distance $\epsilon$:

$$n \gtrsim \sigma^2/(c_1\epsilon)^2 \qquad \text{(the convex-row } N\text{, without the } 1/\delta\text{)}.$$

4. **Flat** ($c_1$ small) or $\|\nabla J\| \to 0$: the noise dominates and $n$ becomes intractable.

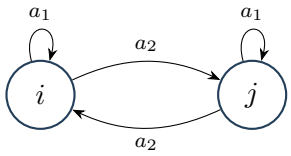
> The theory's $1/c_1^2$ *is* the signal-to-noise cost. The next two MDPs have an (almost) zero gradient: this picture, in RL.

## 4.5 Example 1: exploration




- Two actions move left, one moves right. Uniform random policy: expected time to the goal $= 3(2^n - n - 1) \approx 10^{15}$ for $n = 50$.
- Before reaching the goal every sensible gradient estimate is $\mathbf{0}$: the right *magnitude*, but **no direction**.
- Importance sampling does not help: the weights would be exponentially large.

## 4.6 Example 2: plateaus




Gibbs policy: $\pi(a; s) \propto \exp(\theta_{sa})$

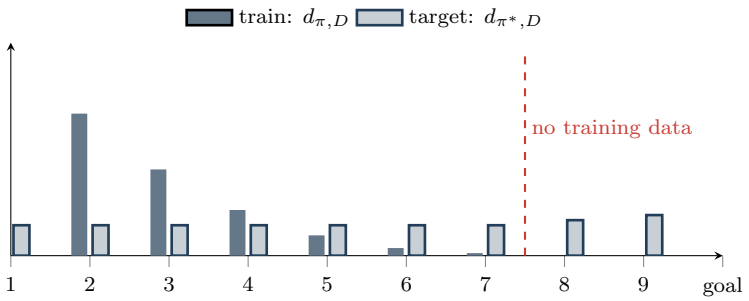
1. Start: $\rho(i) = 0.8$, $\rho(j) = 0.2$ $(\pi(a_1; i) = 0.8, \pi(a_1; j) = 0.9)$.
2. Raising the self-loop at $i$ **starves** state $j$.
3. $\rho(j)$ drops to $\approx 10^{-7} \Rightarrow$ nothing is learned at $j$.
4. Result: an extremely **flat plateau** (on a $10^7$ time scale).

**Root cause**

$\eta_D$ weighs improvements by $d_{\pi,D}$: it is **insensitive to improvement at unlikely states** — unlikely under the *current* policy — even when those states are needed for near-optimal performance.

**Source:** K&L Sec. 3.2, Figs. 3.1–3.3.

## 4.7 Gap 2 as covariate shift



**Train**

Samples come from where the **current** $\pi$ goes.

**Target**

Near-optimal play needs good actions where $\pi^*$ goes.

**No overlap**

Zero gradient (Ex. 1), starved state $j$ (Ex. 2).

**Unlike standard covariate shift:**

1. the target $d_{\pi^*,D}$ needs $\pi^*$ — **unknown**;
2. the data come from $\pi$ itself — the gap can **grow** as $\pi$ learns (Ex. 2);
3. the "labels" $A_\pi$ change with every update;
4. the off-policy fix is a **product** of per-step ratios $\prod_t \pi/q$ — exponential in the horizon, not a single ratio $p_{\text{test}}/p_{\text{train}}$.

**Instead**

Change *where the data come from*: train from a restart distribution $\mu$, still judged by $\eta_D$. The precise version (a density ratio) is Thm. 6.2.

> Related: imitation learning (DAgger) — the learner's own mistakes change which states it visits. Augmentation view: $\mu$-restarts show the learner states it would not reach; unlike classic augmentation, the states are real and the "labels" $\hat{Q}_\pi$ come from real rollouts. The price is reset access.

## 5 The two fixes

**1. Conservative update → Gap 1**

Don't jump to the greedy policy — **mix**:

$$\pi_{\text{new}}(a;s) = (1-\alpha)\,\pi(a;s) + \alpha\,\pi'(a;s)$$

- $\alpha \in [0,1]$ acts as a **step size**.
- $\alpha = 1$ is greedy policy iteration.

**2. Restart distribution $\mu$ → Gap 2**

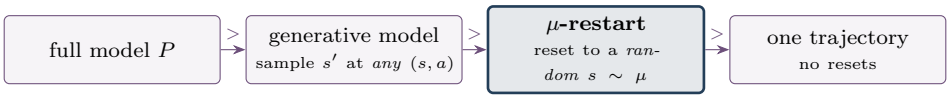
Optimize a measure that weighs states more uniformly:

$$\eta_\mu(\pi) = \mathbb{E}_{s\sim\mu}\big[V_\pi(s)\big]$$

- $\mu$ is "exploratory"; the agent can restart from $s \sim \mu$.
- $D$: deployment (given). $\mu$: training resets (chosen).

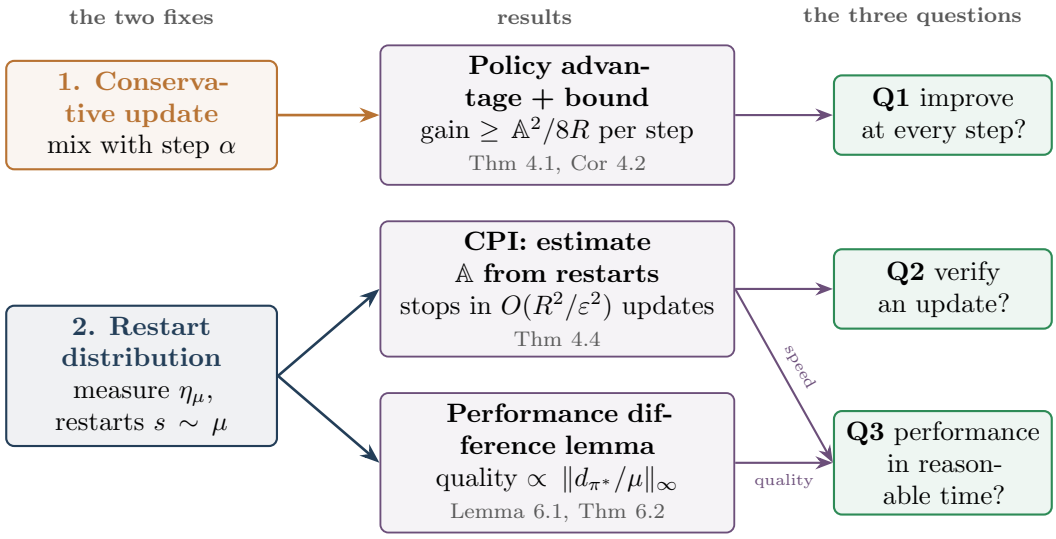
- Any **optimal** policy maximizes both $\eta_\mu$ and $\eta_D$ — but within a restricted class they can differ.
- Sec. 7.2 of the paper: train under $\tilde{\mu} = (1-\beta)\mu + \beta D$ to keep some weight on $D$.

**How strong is a $\mu$-restart?** Access to the environment, from strongest to weakest:




Generative model: Kearns, Mansour & Ng (1999). $\mu$-restarts need training-time reset access, e.g. a simulator, or a lab where the start state can be set.

### 5.1 Roadmap: from the two fixes to the three answers




**Key idea**

$\mu$ sets the **quality** of the result, not the **speed**: the number of updates does not depend on $\mu$.

## 6 The performance difference lemma

**Definition: per-state advantage of $\pi'$ over $\pi$**

One step of $\pi'$ at $s$, then back to $\pi$:

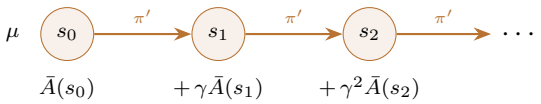
$$\bar{A}(s) = \mathbb{E}_{a\sim\pi'(\cdot;s)}\big[A_\pi(s,a)\big]$$

**Lemma 6.1 (performance difference)**

For any $\pi, \pi'$ and start distribution $\mu$:

$$\eta_\mu(\pi') - \eta_\mu(\pi) \;=\; \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d_{\pi',\mu}}\big[\bar{A}(s)\big] \;=\; \mathbb{E}\Big[\textstyle\sum_t \gamma^t\,\bar{A}(s_t)\;\Big|\; s_0\sim\mu,\ \pi'\Big]$$

The second equality: App. C.




**Exact**

The gain is the one-step advantages, collected along the **new** policy's path.

**Not measurable**

It needs $d_{\pi',\mu}$: the states of a policy we have not run yet.

**Proof sketch.**

1. Along a trajectory of $\pi'$, add and subtract $V_\pi(s_t)$ at each step.
2. The terms $\gamma V_\pi(s_{t+1}) - V_\pi(s_t)$ telescope to $-V_\pi(s_0)$.
3. What remains is $\sum_t \gamma^t A_\pi(s_t, a_t)$; average over $a_t \sim \pi'$.

In operator form, $\bar{A}(s) = (T_{\pi'}V_\pi)(s) - V_\pi(s)$: what we gain by deviating to $\pi'$ for one step at $s$. This is also the starting point of TRPO (its Eq. 1).

# 7 The policy advantage

**Definition: policy advantage (K&L Sec. 4.1)**

$$\mathbb{A}_{\pi,\mu}(\pi') = \mathbb{E}_{s\sim d_{\pi,\mu}}\Big[\mathbb{E}_{a\sim\pi'(\cdot;s)}\big[A_\pi(s,a)\big]\Big] = \mathbb{E}_{s\sim d_{\pi,\mu}}\big[\bar{A}(s)\big]$$

1. Lemma 6.1's average, but on the states the **old** $\pi$ visits (from $\mu$) $\Rightarrow$ **measurable**.
2. One step of policy improvement maximizes it.
3. It is exactly the **first-order** change of $\eta_\mu$ in $\alpha$ (proof: App. A):

$$\left.\frac{\partial\eta_\mu}{\partial\alpha}\right|_{\alpha=0} = \frac{1}{1-\gamma}\mathbb{A}_{\pi,\mu}(\pi') \qquad\Longrightarrow\qquad \Delta\eta_\mu = \frac{\alpha}{1-\gamma}\mathbb{A}_{\pi,\mu}(\pi') + O(\alpha^2)$$

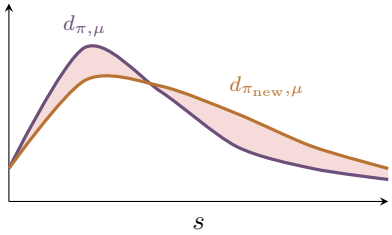
**Key idea**

Small $\alpha$ + positive policy advantage $\Rightarrow$ improvement. Large $\alpha \Rightarrow$ possible degradation. **How large can $\alpha$ be?**
For TRPO: a surrogate built on the *old* state distribution matches the true objective to first order.

## 7.1 The exact change vs. the policy advantage

Apply Lemma 6.1 to the mixture $\pi_{\text{new}} = (1-\alpha)\pi + \alpha\pi'$. Per state, $\mathbb{E}_{a\sim\pi_{\text{new}}}A_\pi(s,a) = (1-\alpha)\cdot 0 + \alpha\bar{A}(s) = \alpha\bar{A}(s)$. Then add and subtract the old distribution:

$$\begin{aligned}(1-\gamma)\big(\eta_\mu(\pi_{\text{new}}) - \eta_\mu(\pi)\big) &= \alpha\textstyle\sum_s d_{\pi_{\text{new}},\mu}(s)\,\bar{A}(s)\\ = \underbrace{\alpha\textstyle\sum_s d_{\pi,\mu}(s)\,\bar{A}(s)}_{=\alpha\mathbb{A}\text{: old states, measurable}} &+ \underbrace{\alpha\textstyle\sum_s \big(d_{\pi_{\text{new}},\mu} - d_{\pi,\mu}\big)(s)\,\bar{A}(s)}_{\text{gap: state shift, not measurable}}\end{aligned}$$




shaded: the shift — $\bar{A}$ unknown there.

- $\alpha\mathbb{A}$: the first-order term above.
- Gap: **two factors of** $\alpha$ — a smaller $\bar{A}$ *and* less shift ($\pi_{\text{new}}$ deviates from $\pi$ only w.p. $\alpha$ per step) $\Rightarrow O(\alpha^2)$.
- Thm. 4.1 is a **lower bound on the gap**: a worst case on the shaded mass.

**Key idea**

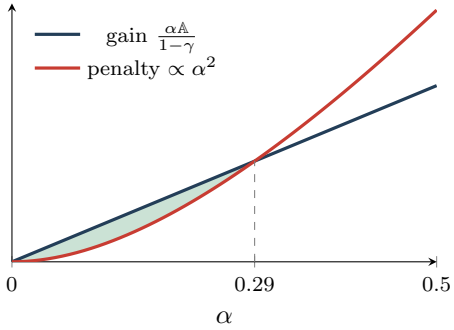
Exact but unmeasurable (Lemma 6.1) = measurable ($\alpha\mathbb{A}$) + gap. We only need to **bound the gap**.

# 8 How big a step? The improvement bound

**Theorem 4.1 (K&L)**

With $\varepsilon = \max_s \left|\mathbb{E}_{a\sim\pi'}[A_\pi(s,a)]\right|$, for all $\alpha \in [0,1]$:

$$\eta_\mu(\pi_{\text{new}}) - \eta_\mu(\pi) \;\geq\; \frac{\alpha}{1-\gamma}\Big(\mathbb{A} - \frac{2\alpha\gamma\varepsilon}{1-\gamma(1-\alpha)}\Big)$$




Illustrative: $\gamma = 0.6$, $\varepsilon = 1$, $\mathbb{A} = 0.6$. Green: improvement guaranteed.

- **Gain** $\alpha\mathbb{A}$: linear, measured on old states.
- **Penalty**: worst case of the state shift, $\propto \alpha^2$ near 0.
- **Small** $\alpha$: the gain wins $\Rightarrow$ guaranteed progress whenever $\mathbb{A} > 0$.
- $\alpha = 1$: the B&T penalty plus a gain; usually loses (needs $\gamma < \frac{1}{2}$). Comparison: App. B.

## 8.1 Choosing the step: Corollary 4.2

Three steps.

**Step 1 — simplify.** Lower the bound further so that only the measurable $\mathbb{A}$ is left. Use $\varepsilon \leq R$, $1-\gamma(1-\alpha) \geq 1-\gamma$, $\gamma \leq 1$:

$$\eta_\mu(\pi_{\text{new}}) - \eta_\mu(\pi) \;\geq\; \frac{\alpha}{1-\gamma}\Big(\mathbb{A} - \frac{2\alpha R}{1-\gamma}\Big) \;=:\; f(\alpha)$$

**Step 2 — maximize.** $f(\alpha) = \frac{\mathbb{A}}{1-\gamma}\alpha - \frac{2R}{(1-\gamma)^2}\alpha^2$ is a downward parabola:

$$f'(\alpha) = \frac{\mathbb{A}}{1-\gamma} - \frac{4R\,\alpha}{(1-\gamma)^2} = 0 \quad\Longrightarrow\quad \alpha^* = \frac{(1-\gamma)\mathbb{A}}{4R}$$

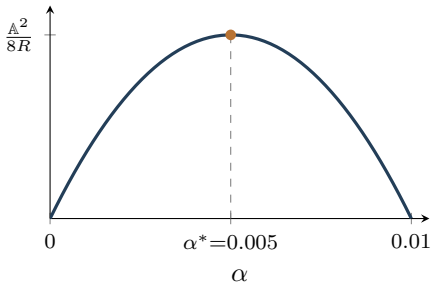
**Step 3 — plug in.**

$$f(\alpha^*) = \frac{\mathbb{A}}{1-\gamma}\cdot\frac{(1-\gamma)\mathbb{A}}{4R} - \frac{2R}{(1-\gamma)^2}\cdot\frac{(1-\gamma)^2\mathbb{A}^2}{16R^2} = \frac{\mathbb{A}^2}{4R} - \frac{\mathbb{A}^2}{8R} = \frac{\mathbb{A}^2}{8R}$$

**Corollary 4.2**

If $\mathbb{A} \geq 0$, then $\alpha = \frac{(1-\gamma)\mathbb{A}}{4R}$ guarantees

$$\eta_\mu(\pi_{\text{new}}) - \eta_\mu(\pi) \geq \frac{\mathbb{A}^2}{8R}.$$




$f(\alpha)$ for $\gamma = 0.9$, $R = 1$, $\mathbb{A} = 0.2$.

**Computable**

$\mathbb{A}$ averages over the *old* policy's states ($\mu$-restarts); $R$ is known. The unknown $\varepsilon$ (a max over all states) is gone.

**Tiny steps**

$\mathbb{A} \leq R$ gives $\alpha \leq (1-\gamma)/4$ ( $\leq 0.0025$ at $\gamma = 0.99$), and $\alpha \to 0$ as $\mathbb{A} \to 0$. This worst-case conservatism is what TRPO relaxes.

# 9 The CPI algorithm

**Algorithm: Conservative Policy Iteration**

1. $\pi' \leftarrow G_\varepsilon(\pi, \mu)$ (greedy policy chooser)
2. Estimate $\hat{\mathbb{A}} \approx \mathbb{A}_{\pi,\mu}(\pi')$ to $\pm\frac{\varepsilon}{3}$ with $O\big(\frac{R^2}{\varepsilon^2}\log\frac{R^2}{\delta\varepsilon^2}\big)$ $\mu$-restarts.
3. If $\hat{\mathbb{A}} < \frac{2\varepsilon}{3}$: **stop**, return $\pi$.
4. Else: mix with $\alpha = \frac{(1-\gamma)(\hat{\mathbb{A}}-\varepsilon/3)}{4R}$, and go to 1.

**Definition: greedy policy chooser $G_\varepsilon$ and OPT**

$G_\varepsilon(\pi,\mu)$ returns $\pi'$ with $\mathbb{A}_{\pi,\mu}(\pi') \geq \mathrm{OPT}(\mathbb{A}_{\pi,\mu}) - \varepsilon$, where

$$\mathrm{OPT}(\mathbb{A}_{\pi,\mu}) \equiv \max_{\pi'} \mathbb{A}_{\pi,\mu}(\pi') = \mathbb{E}_{s\sim d_{\pi,\mu}}\big[\max_a A_\pi(s,a)\big]$$

is the largest policy advantage any $\pi'$ can achieve (Def. 4.3). Small OPT $\Rightarrow$ no policy improves much on $\pi$ under $d_{\pi,\mu}$. One way to build $G_\varepsilon$: a regression on advantages with **average** error $\leq \varepsilon/2$ (App. G).

**Theorem 4.4 (K&L)**

With probability $\geq 1-\delta$, CPI:

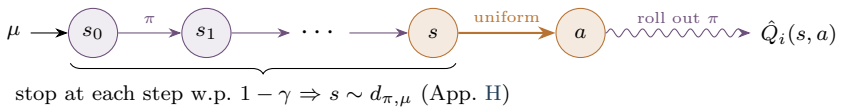
i) improves $\eta_\mu$ at **every** update;
ii) stops within $72\,R^2/\varepsilon^2$ calls to $G_\varepsilon$;
iii) returns $\pi$ with $\mathrm{OPT}(\mathbb{A}_{\pi,\mu}) < 2\varepsilon$.

> Why at most $72R^2/\varepsilon^2$ updates: each update gains $\geq \mathbb{A}^2/8R \geq \varepsilon^2/(72R)$ (since $\mathbb{A} \geq \varepsilon/3$ when CPI steps), and $\eta_\mu \leq R$. The count does **not** depend on $\mu$ or on the size of the state space.

## 9.1 Verifying an update: estimating $\mathbb{A}$ from $\mu$-restarts

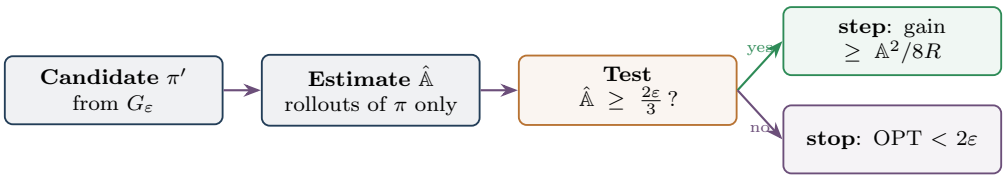
Since $V_\pi(s) = \sum_a \pi(a;s)\,Q_\pi(s,a)$, the policy advantage needs only $Q_\pi$:

$$\mathbb{A}_{\pi,\mu}(\pi') = \mathbb{E}_{s\sim d_{\pi,\mu}}\Big[\sum_a \big(\pi'(a;s) - \pi(a;s)\big)\,Q_\pi(s,a)\Big]$$




$$x_i = n_a\,\hat{Q}_i(s,a)\big(\pi'(a;s) - \pi(a;s)\big) \in [-n_aR,\, n_aR], \qquad \hat{\mathbb{A}} = \tfrac{1}{k}\textstyle\sum_{i=1}^k x_i$$
$$\text{Hoeffding:}\quad \Pr\big(|\hat{\mathbb{A}} - \mathbb{A}| > \Delta\big) \;\leq\; 2\,e^{-k\Delta^2/(2n_a^2R^2)}$$

1. **Unbiased:** $a$ uniform $\Rightarrow \mathbb{E}_a[n_a f(a)] = \sum_a f(a)$. An importance weight $n_a$ (the number of actions) — App. J, App. I.
2. **Only rollouts of the current** $\pi$ — no need to run $\pi_{\text{new}}$. **This is the answer to Q2.**
3. CPI uses $\Delta = \varepsilon/3$, so $\hat{\mathbb{A}} - \frac{\varepsilon}{3} \leq \mathbb{A}$ is safe for the stop test and for $\alpha$.




**The naive check** — run $\pi'$ and compare returns — is noisy and needs rollouts of the **new** policy ($d_{\pi',\mu}$, Lemma 6.1). CPI tests *before* stepping, with data from the current policy only.

## 10 How good is the policy found?

**Theorem 6.2 (K&L)**

If $\mathrm{OPT}(\mathbb{A}_{\pi,\mu}) < \varepsilon$, then for any $\tilde{\mu}$:

$$\eta_{\tilde{\mu}}(\pi^*) - \eta_{\tilde{\mu}}(\pi) \;\leq\; \frac{\varepsilon}{1-\gamma} \left\| \frac{d_{\pi^*,\tilde{\mu}}}{d_{\pi,\mu}} \right\|_\infty \;\leq\; \frac{\varepsilon}{(1-\gamma)^2} \left\| \frac{d_{\pi^*,\tilde{\mu}}}{\mu} \right\|_\infty$$

With $\tilde{\mu} = D$ this bounds the real goal $\eta_D$ (Cor. 4.5).

**Proof in three steps.**

1. Lemma 6.1 for $\pi^*$ vs. $\pi$: $\eta(\pi^*) - \eta(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{d_{\pi^*,\tilde{\mu}}} \left[ \mathbb{E}_{a\sim\pi^*} A_\pi \right]$.
2. $\mathbb{E}_{a\sim\pi^*}[A_\pi] \leq \max_a A_\pi$, which is $\geq 0$ (since $\mathbb{E}_{a\sim\pi} A_\pi = 0$).
3. Change of measure: $\mathbb{E}_p[f] \leq \|p/q\|_\infty \mathbb{E}_q[f]$ for $f \geq 0$, with $\mathbb{E}_{d_{\pi,\mu}}[\max_a A_\pi] = \mathrm{OPT}(\mathbb{A}_{\pi,\mu}) < \varepsilon$.

**A covariate-shift bound**

test risk $\leq$ density ratio $\times$ train risk

- train risk: $\mathrm{OPT}(\mathbb{A}_{\pi,\mu})$, on CPI's data $d_{\pi,\mu}$
- test: the states $\pi^*$ visits, $d_{\pi^*,\tilde{\mu}}$

**Uniform $\mu$ caps the ratio**

$d_{\pi,\mu} \geq (1-\gamma)\mu$, so with uniform $\mu$ the ratio is $\leq |S|/(1-\gamma)$ — for **any** $\pi^*$. Later RL theory calls it the *concentrability coefficient*.

Two factors of $\frac{1}{1-\gamma}$: one from the lemma, one from $d_{\pi,\mu} \geq (1-\gamma)\mu$. Sec. 7.3 of the paper: choose $\mu$ using prior knowledge of where an optimal policy goes.

## 11 Summary of Part I

**CPI's answers to the three questions**

**Q1 Improving measure:** $\eta_\mu$ improves at every update (Thm. 4.4).
**Q2 Verification:** estimate the policy advantage $\mathbb{A}$ from samples of the current policy (Hoeffding).
**Q3 Performance:** within $O(R^2/\varepsilon^2)$ updates, close to optimal up to the mismatch factor (Thm. 6.2).

**Kept going forward**

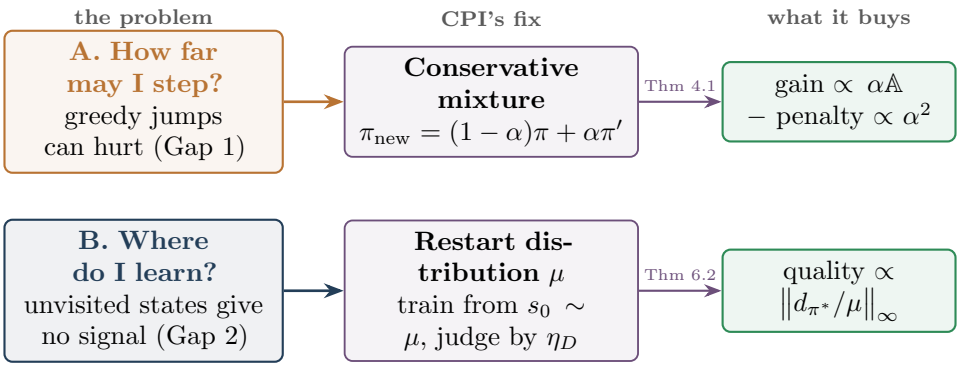
- Surrogate on the **old** state distribution
- Improvement lower bound = gain − penalty
- Small, controlled steps

**Limitations**

- Only **mixture** policies
- Needs a restart distribution
- Theoretical step sizes are tiny

Like non-convex SGD (a bounded objective and a sure gain per step ⇒ few steps to a stationary point), but Thm. 6.2 turns "stationary" into "near-optimal". Details: App. F.

### 11.1 The two lanes to follow




**Key idea**

Two **separate** axes: **update size** (A) and **state coverage** (B). TRPO keeps lane A and drops lane B; GAE and PPO never bring B back.

## Part II

# Trust Region Policy Optimization

> **The question of Part II:** CPI's guarantee only holds for *mixture* policies. TRPO replaces the mixing weight $\alpha$ by a **distance between policies**, gets a monotone-improvement bound for **any** stochastic policy $\pi_\theta$, and then makes it practical in **five simplifications**. It keeps lane A of CPI and drops lane B.

## 12 Setting: what changes from CPI

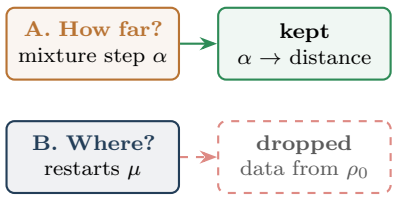
**Trust Region Policy Optimization**
Schulman, Levine, Moritz, Jordan & Abbeel · ICML 2015

**Introduces:** TRPO — CPI's guarantee for any stochastic policy $\pi_\theta$
Local copy: `trust_region_policy_optimization.md`

An MDP $(\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma)$:

1. **Finite** $\mathcal{S}, \mathcal{A}$ in the proofs — also holds for **continuous** spaces (conditions in App. K).
2. **Discounted**: $\gamma \in (0, 1)$, reward $r(s)$.
3. **Unknown dynamics**: sample trajectories only.
4. **Start distribution** $\rho_0$ (CPI's $D$) — **no restart distribution** $\mu$.
5. **Unnormalized**: $\eta(\pi) = \mathbb{E}\big[\sum_t \gamma^t r(s_t)\big]$, $s_0 \sim \rho_0$.
6. **Any** differentiable stochastic policy $\pi_\theta$ (e.g. a neural net).

| | CPI | TRPO |
|---|---|---|
| policies | mixtures $\pi_\alpha$ | any $\pi_\theta$ |
| new candidate | greedy chooser $G_\varepsilon$ | a step in $\theta$ |
| "how far" | weight $\alpha$ | distance $D(\pi_{\text{old}}, \pi_{\text{new}})$ |
| start states | $D$, trained from $\mu$ | $\rho_0$ only |
| scale | normalized by $1-\gamma$ | unnormalized |




**Lane B is dropped**

The only resets in TRPO are in the "vine" scheme (§16.5), and they go to states *already reached* from $\rho_0$ — not an exploratory $\mu$. We return to this in §18.

**Roadmap of the part:** same identity → Theorem 1 (any policy) → KL and MM → five practical simplifications → the algorithm.

## 13 The same identity, a local surrogate

Unnormalized return and visitation:

$$\eta(\pi) = \mathbb{E}\Big[\sum_t \gamma^t r(s_t)\Big], \qquad \rho_\pi(s) = \sum_t \gamma^t P(s_t = s).$$

**Performance difference (TRPO Eq. 2 = K&L Lemma 6.1)**

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a \mid s)\, A_\pi(s,a)$$

$\rho_{\tilde{\pi}}$ depends on $\tilde{\pi}$ in a complicated way $\Rightarrow$ replace it by the **old** visitation:

**Definition: local surrogate $L_\pi$ (TRPO Eq. 3)**

$$L_\pi(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_\pi(s) \sum_a \tilde{\pi}(a \mid s)\, A_\pi(s,a)$$

**Matches $\eta$ to first order**

$L_{\pi_{\theta_0}}(\pi_{\theta_0}) = \eta(\pi_{\theta_0})$ and the gradients are equal at $\theta_0$. The same idea as CPI's policy advantage, now for any $\theta$.

**Says nothing about step size**

First-order agreement says a *small enough* step helps — not **how big** a step is safe.

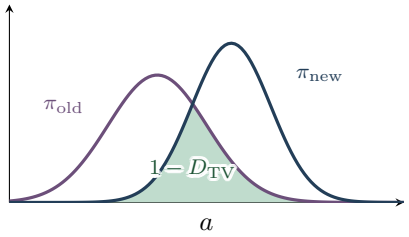
TRPO restates CPI's bound (slightly weakened): $\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{2\epsilon\gamma}{(1-\gamma)^2}\alpha^2$ with $\epsilon = \max_s |\mathbb{E}_{a\sim\pi'} A_\pi(s,a)|$ — valid only for mixtures. Why the first-order statement holds for any parameterization: App. A (the proof never uses that $\alpha$ is a mixing weight).

## 14 From mixtures to any stochastic policy: Theorem 1

Replace the mixing weight $\alpha$ by the **total variation distance**:

**Definition: total variation, max over states**

$$D_{\text{TV}}(p \,\|\, q) = \tfrac{1}{2}\textstyle\sum_a \big|p(a) - q(a)\big|, \qquad D_{\text{TV}}^{\max}(\pi, \tilde{\pi}) = \max_s D_{\text{TV}}\big(\pi(\cdot \mid s) \,\|\, \tilde{\pi}(\cdot \mid s)\big)$$




- Green area: the **shared mass** $\sum_a \min(p,q) = 1 - D_{\text{TV}}$.
- **Coupling**: draw $a \sim \pi_{\text{old}}$, $\tilde{a} \sim \pi_{\text{new}}$ so that they are **equal** with probability $1 - D_{\text{TV}}$ (maximal coupling).
- So the new policy "acts like the old one" w.p. $1 - \alpha$ at each state — exactly the structure of a CPI mixture.

**Theorem 1 (TRPO)**

Let $\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$. Then

$$\eta(\pi_{\text{new}}) \;\geq\; L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2}\,\alpha^2, \qquad \epsilon = \max_{s,a} |A_\pi(s,a)|$$

1. **Proof idea**: couple the two policies; they agree w.p. $1 - \alpha$, which is all the CPI proof needs (TRPO App. A, adapted from K&L Thm. 4.1; App. B gives a second proof by perturbation theory).
2. **Different** $\epsilon$: a max over $(s,a)$ of $|A_\pi|$, not CPI's max over $s$ of an expectation. How the two relate: App. B.
3. **Why a max over states**: the coupling must hold at every state the new policy might reach.

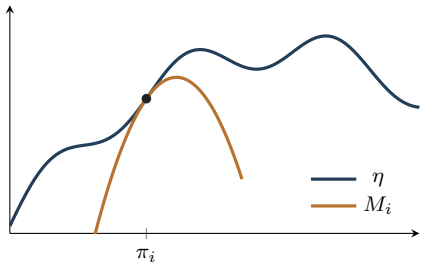
# 15 From TV to KL: a minorize–maximize algorithm

**Definition: KL divergence**

$$D_{\mathrm{KL}}(p\|q) = \sum_a p(a) \log \tfrac{p(a)}{q(a)} \;\geq 0 \qquad \text{— smooth in } \theta, \text{ unlike } D_{\mathrm{TV}}.$$

Use $D_{\mathrm{TV}}(p\|q)^2 \leq D_{\mathrm{KL}}(p\|q)$ (Pinsker-type) in Theorem 1:

$$\eta(\tilde{\pi}) \;\geq\; \underbrace{L_\pi(\tilde{\pi}) - C\, D_{\mathrm{KL}}^{\max}(\pi, \tilde{\pi})}_{M_i(\tilde{\pi})}, \qquad C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$$




Illustrative. $M_i$ touches $\eta$ at $\pi_i$ and lies below it.

1. $M_i \leq \eta$ everywhere, with equality at $\pi_i$.
2. Maximize $M_i$:

$$\eta(\pi_{i+1}) - \eta(\pi_i) \;\geq\; M_i(\pi_{i+1}) - M_i(\pi_i) \;\geq\; 0.$$

3. $\Rightarrow$ **monotone improvement** (with exact advantages).

MM = minorization–maximization (Hunter & Lange 2004), the same family as EM.

**Key idea**

This penalized problem is what **the theory covers**. Everything after this point trades a guarantee for practicality.

# 16 Five practical simplifications

Each simplification is shown with the same three boxes: why it is needed, what changes, and what it costs.

## 16.1 1. From penalty to constraint: the trust region

**1. penalty → constraint** → 2. max → mean KL → 3. samples + IS → 4. linear / quadratic → 5. CG + line search

**Theory (penalty)**

$$\max_\theta \; L_{\theta_{\mathrm{old}}}(\theta) - C\, D_{\mathrm{KL}}^{\max}(\theta_{\mathrm{old}}, \theta)$$

With the theoretical $C$ the steps are **tiny**, and a good $C$ is hard to choose robustly.

**Practice (constraint)**

$$\max_\theta \; L_{\theta_{\mathrm{old}}}(\theta) \quad \text{s.t.} \quad D_{\mathrm{KL}}^{\max}(\theta_{\mathrm{old}}, \theta) \leq \delta$$

$\delta$ lives in policy space, so it transfers across problems ($\delta = 0.01$ in all experiments).

**Why needed**

$C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$: at $\gamma = 0.99$, $C \approx 4 \cdot 10^4\, \epsilon \Rightarrow$ tiny steps.

**What changes**

A hard limit $\delta$ on the KL replaces the penalty $C$: optimize $L$ only where we *trust* it.

**What it costs**

The guarantee becomes a heuristic; $\delta$ is a hyperparameter.

The constraint is the Lagrangian-dual view of the penalty: each $\delta$ corresponds to some $C$.

## 16.2 2. From max KL to mean KL

1. penalty → constraint → **2. max → mean KL** → 3. samples + IS → 4. linear / quadratic → 5. CG + line search

Replace the max over states by an average under the old visitation:

$$\bar{D}^{\rho}_{\mathrm{KL}}(\theta_1, \theta_2) = \mathbb{E}_{s\sim\rho}\Big[D_{\mathrm{KL}}\big(\pi_{\theta_1}(\cdot \mid s) \,\|\, \pi_{\theta_2}(\cdot \mid s)\big)\Big]$$

$$\max_{\theta} \; L_{\theta_{\mathrm{old}}}(\theta) \quad \text{s.t.} \quad \bar{D}^{\rho_{\theta_{\mathrm{old}}}}_{\mathrm{KL}}(\theta_{\mathrm{old}}, \theta) \le \delta$$

**Why needed**

$D^{\max}_{\mathrm{KL}}$ is one constraint **per state**, also at states we never sample.

**What changes**

An **expectation** over visited states ⇒ estimated from the same rollouts.

**What it costs**

Rarely visited states may change a lot. Empirically close to max KL (cart-pole, §19).

## 16.3 Tool: importance sampling

1. **Monte Carlo:** an expectation ≈ an average of samples, $\mathbb{E}_{a\sim p}[f] \approx \frac{1}{n}\sum_i f(a_i)$, $a_i \sim p$.
2. **Samples only from $q$?** Weight by $w = p/q$ (needs $q > 0$ where $p > 0$):

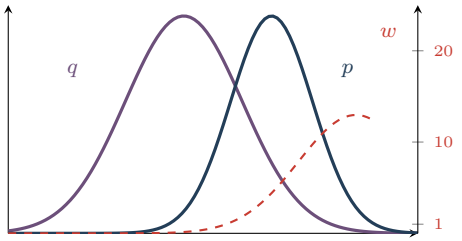
$$\mathbb{E}_{a\sim p}[f] = \textstyle\sum_a q\,\frac{p}{q}\,f = \mathbb{E}_{a\sim q}\big[w\,f\big]$$

3. **Unbiased for any $q$; the variance depends on $q$:**

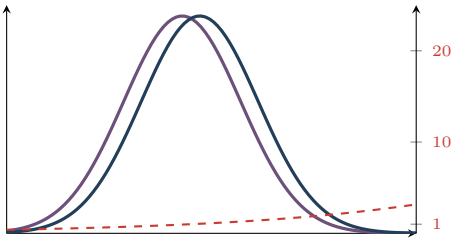
$$\hat{\mu} = \tfrac{1}{n}\textstyle\sum_i w(a_i)\,f(a_i), \qquad \mathrm{Var}[\hat{\mu}] = \tfrac{1}{n}\mathrm{Var}_q\big[wf\big]$$

4. **In TRPO** (single path), at each state $s$: $p = \pi_\theta(\cdot \mid s)$, $q = \pi_{\theta_{\mathrm{old}}}(\cdot \mid s)$, $w = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\mathrm{old}}}(a|s)}$.

$p$ far from $q$: huge rare weights




trust region: all $w \approx 1$




**The trust region does two jobs**

It keeps $L$ accurate **and** it keeps the weights tame. PPO later clips exactly this ratio.

**Only actions are reweighted**

States stay $s \sim \rho_{\theta_{\mathrm{old}}}$. Reweighting states would need $\rho_\theta/\rho_{\theta_{\mathrm{old}}}$, which is unknown — so TRPO cannot run offline on a fixed dataset.

**Link to CPI:** CPI's estimate of $\mathbb{A}$ already used IS with a uniform $q$ and weight $n_a$ (App. J). **Callback to Gap 2:** reweighting whole trajectories gives $w(\tau) = \prod_t \pi(a_t \mid s_t)/\pi_b(a_t \mid s_t)$ — the dynamics cancel, but the product is exponentially large or small; IS only moves the exponential cost into the variance.

## 16.4 3. Turning sums into samples

1. penalty → constraint → 2. max → mean KL → **3. samples + IS** → 4. linear / quadratic → 5. CG + line search

Start from the trust-region problem written with sums:

$$\max_\theta \sum_s \rho_{\theta_{\text{old}}}(s) \sum_a \pi_\theta(a \mid s)\, A_{\theta_{\text{old}}}(s,a)$$

1. $\sum_s \rho_{\theta_{\text{old}}}(s)[\cdot] \;\to\; \frac{1}{1-\gamma}\mathbb{E}_{s\sim\rho_{\theta_{\text{old}}}}[\cdot]$ — visited states
2. $A_{\theta_{\text{old}}} \;\to\; Q_{\theta_{\text{old}}}$ — shifts the objective by a constant: $\sum_a \pi_\theta V_{\theta_{\text{old}}} = V_{\theta_{\text{old}}}$
3. $\sum_a \;\to$ importance sampling with a sampling distribution $q$ — §16.3

$$\max_\theta \; \mathbb{E}_{s\sim\rho_{\theta_{\text{old}}},\, a\sim q}\left[\frac{\pi_\theta(a \mid s)}{q(a \mid s)}\, Q_{\theta_{\text{old}}}(s,a)\right] \quad \text{s.t.} \quad \mathbb{E}_{s\sim\rho_{\theta_{\text{old}}}}\left[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s))\right] \le \delta$$

**Why needed**

Sums over all $s$ and $a$ need the model; continuous $a$ has no sum at all.

**What changes**

Averages over rollouts of $\pi_{\text{old}}$; the ratio $\pi_\theta/q$; $\hat{Q}$ from returns.

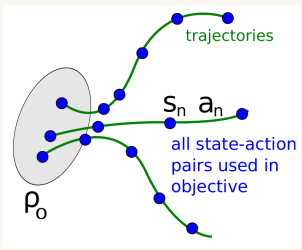
**What it costs**

Noise (variance) in $L$ and its gradient; states are still only $s \sim \rho_{\theta_{\text{old}}}$.

The variance of $\hat{Q}$ is what GAE attacks (Part III).

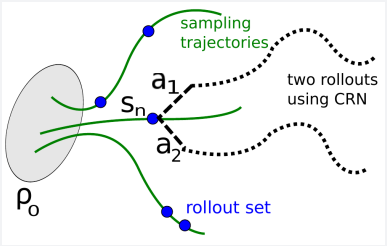
## 16.5 Two sampling schemes

**Single path**




- Roll out $\pi_{\theta_{\text{old}}}$; use **every** $(s_n, a_n)$.
- $q = \pi_{\theta_{\text{old}}}$; $\hat{Q}$ = discounted return-to-go.
- **No state resets** — works on real systems.

**Vine**




- From each $s_n$ in a **rollout set**: try $K$ actions $a_k \sim q$, a short rollout each, with common random numbers.
- **Lower-variance** $\hat{Q}$.
- Needs **resets** — but only to states reached from $\rho_0$: **not** CPI's $\mu$.

$q = \pi_{\theta_i}$ for continuous control, uniform for Atari ("can sometimes achieve better exploration" — over actions, at states already visited). Vine with continuous actions uses self-normalized IS, which removes the need for a baseline. Figures: TRPO Fig. 1.

## 16.6 4. Linear objective, quadratic constraint: the search direction

1. penalty → constraint → 2. max → mean KL → 3. samples + IS → **4. linear / quadratic** → 5. CG + line search

Approximate around $\theta_{\text{old}}$:

$$L(\theta) \approx L(\theta_{\text{old}}) + g^\top(\theta - \theta_{\text{old}}), \qquad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \approx \tfrac{1}{2}(\theta - \theta_{\text{old}})^\top A\,(\theta - \theta_{\text{old}})$$

1. $A_{ij} = \frac{\partial^2}{\partial\theta_i \partial\theta_j} \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta)$: the **curvature of the KL** — the **Fisher information matrix**. Computed analytically, not from sampled score functions.
2. The KL has **no linear term**: it is minimized (zero) at $\theta = \theta_{\text{old}}$, so its gradient vanishes there.
3. Solution direction $s \propto A^{-1}g$: the **natural gradient** — steepest ascent when distance is measured in KL, not in $\|\theta\|$.

| Why needed | What changes | What it costs |
|---|---|---|
| $L$ and the KL are nonlinear in $\theta$; an exact constrained solve per batch is too expensive. | First/second-order Taylor models $\Rightarrow$ a closed-form direction $A^{-1}g$. | Valid only locally — the step must be checked (next). |

With $A^{-1}g$ and a *fixed* step size we would get Kakade's (2002) natural policy gradient; TRPO enforces the constraint at each step instead (App. L).

## 16.7 5. Computing the step: conjugate gradient + line search

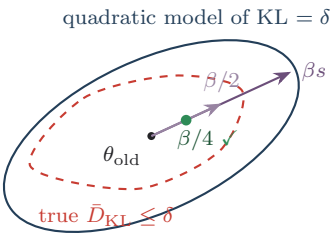



1. **Direction:** solve $A\,s = g$ by **conjugate gradient**. It needs only products $y \mapsto Ay$ (autodiff), $k = 10$ iterations. $A$ has $|\theta|^2$ entries and is never formed.
2. **Largest step to the KL boundary** of the quadratic model:

$$\tfrac{1}{2}(\beta s)^\top A(\beta s) = \delta \;\Rightarrow\; \beta = \sqrt{2\delta / s^\top A s}$$

3. **Backtrack** $\beta,\ \beta/2,\ \beta/4, \ldots$ until

$$L_{\theta_{\text{old}}}(\theta) \text{ improves} \quad \text{and} \quad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \le \delta.$$




Illustrative: the full step and $\beta/2$ fail the true checks, $\beta/4$ passes.

| Why needed | What changes | What it costs |
|---|---|---|
| $A$ cannot even be stored for a neural net; the quadratic model is only local. | CG with Fisher-vector products; a full step to $\delta$; a line search on the true $L$ and KL. | Second order and **one step per batch** — PPO's starting point. |

**Fisher-vector products.** If the network outputs distribution parameters $\mu_\theta(x)$, then $A = J^\top M J$ with $J = \partial\mu/\partial\theta$ and $M$ the analytic Fisher matrix of the distribution w.r.t. $\mu$. $Jy$ and $J^\top v$ are a forward-mode and a backprop pass. FVPs on ∼10% of the data, so one update costs about one gradient. Without the line search, the paper reports occasional large steps with catastrophic drops.

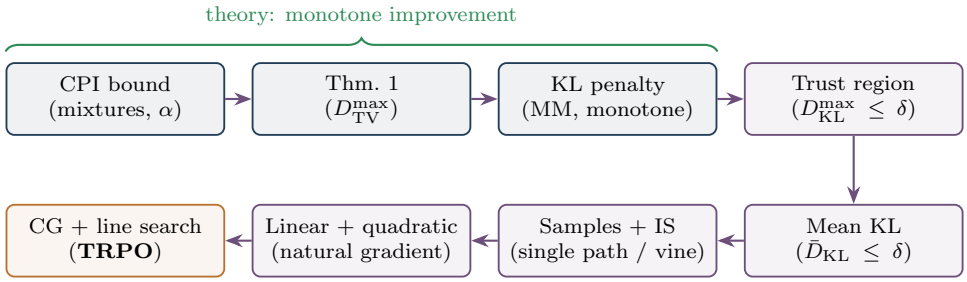
# 17 The TRPO algorithm

**Algorithm: TRPO (one iteration)**

1. Collect $(s, a)$ pairs with Monte Carlo $\hat{Q}$ estimates (single path or vine).
2. Build the sample estimates of the surrogate $L$ and the mean-KL constraint.
3. Compute $g = \nabla_\theta L$; solve $As = g$ with conjugate gradient (Fisher-vector products).
4. Step size $\beta = \sqrt{2\delta / s^\top As}$; backtracking line search on $L$ and the KL.
5. $\theta_{\text{old}} \leftarrow \theta_{\text{old}} + \beta s$.

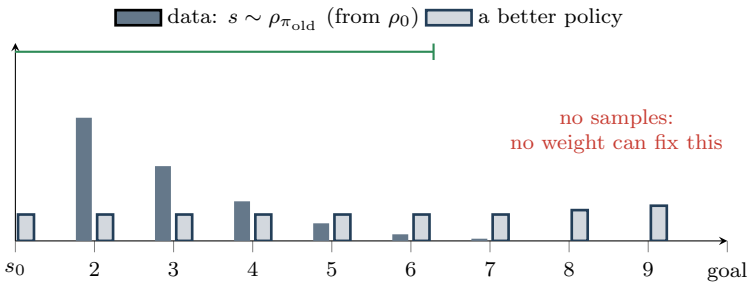
Cost per update: only slightly more than computing the gradient itself.

## 17.1 Recap: from theory to practice




- First three boxes: **theory**. From the trust region on: **practice** — each arrow trades a guarantee for practicality.
- The theory ignores advantage-estimation error (K&L handle it; TRPO omits it for simplicity).
- What is left: noisy $\hat{Q}$ (**GAE**) and a heavy second-order solver (**PPO**).

# 18 What TRPO does not fix: where the data come from




1. **Data:** $s_0 \sim \rho_0$, then $\pi_{\text{old}}$ — CPI's $\mu$ is gone.
2. **Trust region:** limits how much $\pi$ changes **on the states we have**.
3. **Importance sampling:** reweights **actions**, never states.
4. **Vine:** resets only to states already reached from $\rho_0$.

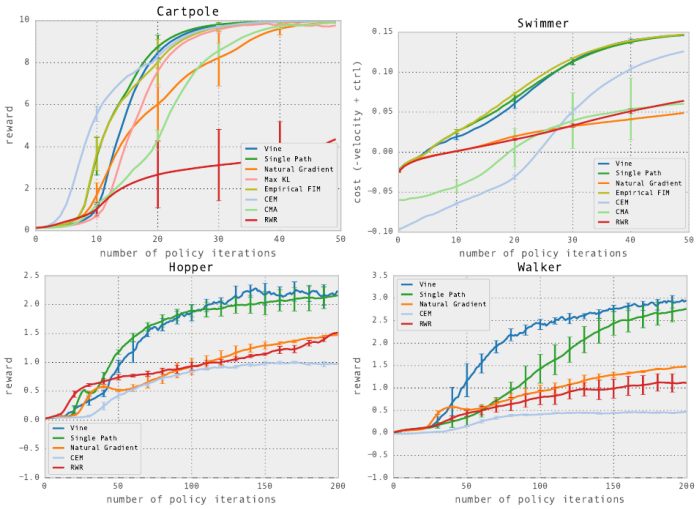
**Trust region $\neq$ exploration**

Lane B (CPI's Gap 2, §4.5) stays open in TRPO, GAE and PPO. The only exploration left is in the actions: a stochastic policy, a uniform $q$ for the vine on Atari, and later PPO's entropy bonus.

Why TRPO still works in the experiments: $\rho_0$ and a stochastic policy already cover the relevant states (locomotion from a standing pose, Atari from the start screen).

# 19 Results

## 19.1 Simulated robots (MuJoCo)




TRPO Fig. 4: learning curves, five runs.

- Swimmer, hopper, walker: torques from joint states; neural-net policies.
- **Vine, single path**: solve all tasks, best scores.
- **Natural gradient** (fixed step): no hopping or walking gait.
- **CEM, CMA** (derivative-free): poor on the larger networks.
- **Max KL** (cart-pole only): a bit slower than mean KL — simplification 2 costs little.
- Same $\delta = 0.01$ in every experiment.

## 19.2 Atari from raw pixels

| | B. Rider | Breakout | Enduro | Pong | Q*bert | Seaquest | S. Invaders |
|---|---|---|---|---|---|---|---|
| Random | 354 | 1.2 | 0 | −20.4 | 157 | 110 | 179 |
| Human | 7456 | 31.0 | 368 | −3.0 | 18900 | 28010 | 3690 |
| Deep Q-learning | 4092 | 168.0 | 470 | 20.0 | 1952 | 1705 | 581 |
| UCC-I (MCTS) | 5702 | 380 | 741 | 21 | 20025 | 2995 | 692 |
| **TRPO single path** | 1425.2 | 10.8 | **534.6** | **20.9** | **1973.5** | **1908.6** | 568.4 |
| **TRPO vine** | 859.5 | 34.2 | 430.8 | **20.9** | **7732.5** | 788.4 | 450.2 |

TRPO Table 1. Bold: TRPO above Deep Q-learning. One run per game.

**Key idea**

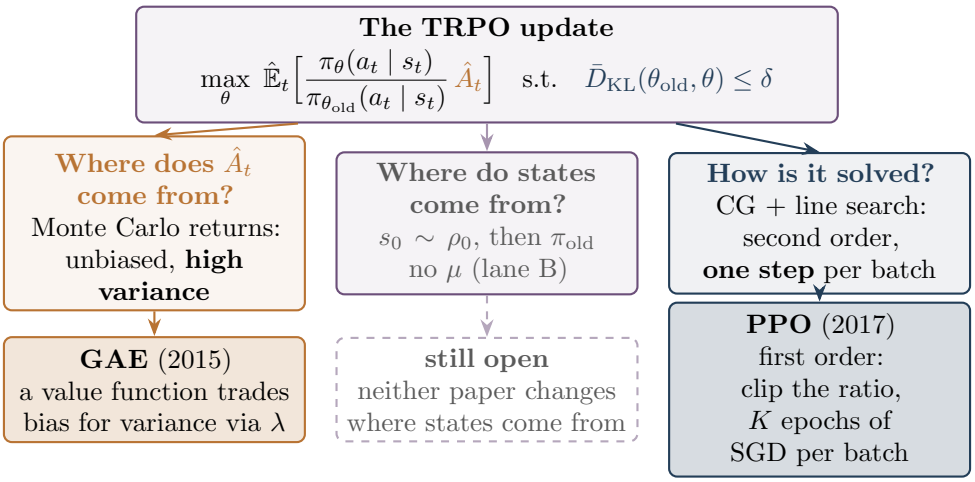
Not built for games — yet the **same** algorithm and settings as for the robots give reasonable scores everywhere. (CNN policy with 33,500 parameters; 500 iterations ≈ 30 hours on 16 cores.)

## Part III

# Generalized Advantage Estimation

> **The question of Part III:** TRPO multiplies each ratio by an advantage estimate $\hat{A}_t$, and Monte Carlo returns make it very noisy. GAE uses a learned **value function** (the critic) and one knob $\lambda$ to trade **bias for variance** — TD($\lambda$), applied to advantages.

## 20 Where TRPO leaves us




The update is written as in the GAE paper (Sec. 6.1, Eq. 31). TRPO itself used Monte Carlo $\hat{Q}$, which differs from an advantage only by a baseline. PPO's intro lists TRPO's other drawbacks: complicated, and incompatible with noise (dropout) or parameter sharing between policy and value.

## 21 Setting

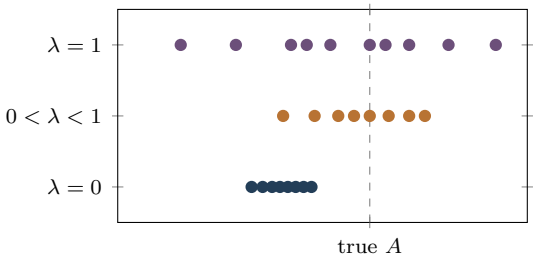
**High-Dimensional Continuous Control Using Generalized Advantage Estimation**
Schulman, Moritz, Levine, Jordan & Abbeel · ICLR 2016 (arXiv 2015)

**Introduces:** GAE($\gamma, \lambda$) — TD($\lambda$) for advantages, plus a trust-region value fit
Local copy: `gae.md`

1. Same MDP as TRPO: start distribution $\rho_0$, unknown dynamics, reward $r_t = r(s_t, a_t, s_{t+1})$.
2. **Undiscounted** goal: maximize $\mathbb{E}\left[\sum_t r_t\right]$ (assumed finite).
3. $\gamma$ is **not part of the problem**: it is an algorithm knob that trades bias for variance.
4. Any policy-gradient method — here TRPO for the policy, and a trust region for $V_\phi$ too.
5. 3D robots, neural nets with $> 10^4$ parameters, torques from raw kinematics.

> All quantities are unnormalized, like TRPO and unlike CPI. The formula itself appeared earlier (Kimura & Kobayashi 1998; Wawrzyński 2009); the paper's contribution is the analysis that makes it usable with batch trust-region methods, plus the trust-region value fit.

### 21.1 What GAE fixes — and what it does not




Each dot: one estimate $\hat{A}_t$ of the same $(s_t, a_t)$ (schematic).

1. TRPO multiplies each ratio by $\hat{A}_t$. From one rollout: **unbiased**, but **very noisy** (top row).
2. A learned **value function** $V$ — the *critic*: low noise, but **biased** if $V$ is wrong (bottom row).
3. $\lambda$ slides between the two: a **bias–variance knob** (middle row).

**Fixes**

Noisy update directions ⇒ fewer samples per step.

**Not an exploration knob**

$\lambda$ changes how we *score* sampled actions, not *which states* we visit.

GAE abstract: two challenges — "the large number of samples typically required, and the difficulty of obtaining stable and steady improvement". Value functions attack the first, a trust region for both policy and value attacks the second. Sec. 1: "bias is more pernicious — even with an unlimited number of samples, bias can cause the algorithm to fail to converge."

## 22 Which weight multiplies the score function?

All policy-gradient estimators have the same form:

$$g = \mathbb{E}\Big[\sum_{t\geq 0} \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]$$

Same expectation for each choice of $\Psi_t$ — **different variance**:

| | $\Psi_t$ | |
|---|---|---|
| 1 | $\sum_{t'\geq 0} r_{t'}$ | total return |
| 2 | $\sum_{t'\geq t} r_{t'}$ | reward after $a_t$ |
| 3 | $\sum_{t'\geq t} r_{t'} - b(s_t)$ | with a baseline |
| 4 | $Q^\pi(s_t, a_t)$ | action value |
| 5 | $\boldsymbol{A^\pi(s_t, a_t)}$ | **advantage** |
| 6 | $r_t + V^\pi(s_{t+1}) - V^\pi(s_t)$ | TD residual |

**Why the advantage**

- "Almost the lowest possible variance" (Greensmith et al., 2004).
- A step along $A \nabla \log \pi$ raises $\pi(a \mid s)$ **iff** $a$ beats the average $V^\pi$ — the gaps of the Notation figure in Part I.

**The discount as a knob**

The discounted gradient $g^\gamma$ uses $A^{\pi,\gamma}$, which ignores effects $\gg \frac{1}{1-\gamma}$ steps later ⇒ less variance, **some bias**. An estimator $\hat{A}_t$ is $\gamma$-*just* if it gives an unbiased estimate of $g^\gamma$ (Def. 1, Prop. 1). From here on, bias means bias *relative to* $g^\gamma$.

## 23 GAE: TD($\lambda$) for advantages

Four steps, from one TD residual to all of them.

1. **TD residual** — one reward, then $V$:

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$$

2. $k$ **residuals** — $\hat{A}_t^{(k)} = \sum_{l<k} \gamma^l \delta_{t+l}^V$ telescopes to

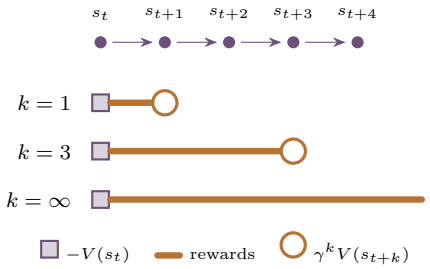
$$\hat{A}_t^{(k)} = -V(s_t) + \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k V(s_{t+k})$$

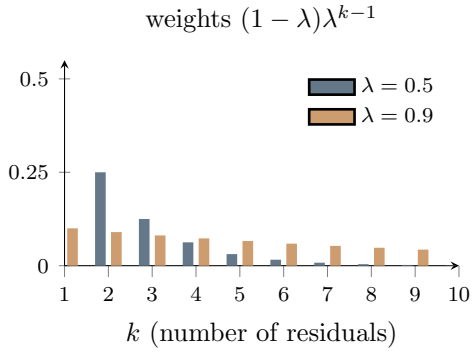
3. **All of them** — the return minus a baseline:

$$\hat{A}_t^{(\infty)} = \sum_{l\geq 0} \gamma^l r_{t+l} - V(s_t)$$

4. **GAE** — average all $k$ with weights $(1-\lambda)\lambda^{k-1}$:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda)\sum_{k\geq 1} \lambda^{k-1} \hat{A}_t^{(k)} = \sum_{l\geq 0} (\gamma\lambda)^l \, \delta_{t+l}^V$$

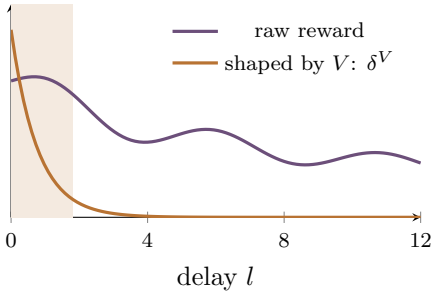



| | $\lambda = 0$: $\hat{A}_t = \delta_t^V$ | $\lambda = 1$: $\hat{A}_t = \sum_l \gamma^l r_{t+l} - V(s_t)$ |
|---|---|---|
| bias | biased unless $V = V^{\pi,\gamma}$ | unbiased for $g^\gamma$, **any** $V$ |
| variance | low | high |

The same picture as TD(0)→Monte Carlo in Part I: TD($\lambda$) estimates a value, GAE an advantage; the only change is the baseline $-V(s_t)$ at the start of every row. The bias shrinks with $k$ because the bootstrapped term $\gamma^k V(s_{t+k})$ is discounted more, while $-V(s_t)$ never adds bias (it is a baseline).

## 24 Two knobs, two kinds of bias




$\chi(l; s, a) = \mathbb{E}[r_{t+l} \mid s, a] - \mathbb{E}[r_{t+l} \mid s]$: how long $a_t$ keeps mattering.

**$\gamma$: cut the horizon**

Drops effects after $\approx \frac{1}{1-\gamma}$ steps. Biased **even with the exact** $V$.

**$\lambda$: shape, then cut again**

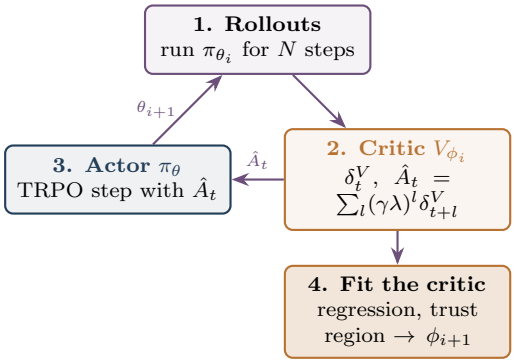
$\delta^V$ is the reward *shaped* by $\Phi = V$: a good $V$ pulls the effect of $a_t$ to $l \approx 0$. Biased **only if** $V$ **is wrong**.

**Key idea**

So the best $\lambda$ is **lower** than the best $\gamma$ (numbers in §26).

**Reward shaping** (Ng et al., 1999): $\tilde{r} = r + \gamma\Phi(s') - \Phi(s)$ leaves $A^{\pi,\gamma}$ unchanged. With $\Phi = V$ it is exactly $\delta^V$, and GAE is the $(\gamma\lambda)$-discounted sum of shaped rewards (Eq. 20). With $\Phi = V^{\pi,\gamma}$ the shaped response is nonzero only at $l = 0$. Response function: $A^{\pi,\gamma}(s,a) = \sum_l \gamma^l \chi(l; s, a)$ (Eq. 22). The curves are illustrative.

## 25 Actor–critic: policy and value networks




- **Actor**: the policy network, updated by TRPO.
- **Critic**: a second network $V_\phi(s) \approx V^{\pi,\gamma}(s)$, fitted by regression to Monte Carlo returns $\hat{V}_n = \sum_l \gamma^l r_{n+l}$.
- **Order matters**: fit $V$ *after* computing $\hat{A}_t$ — an overfit $V$ makes every $\delta_t^V = 0$.

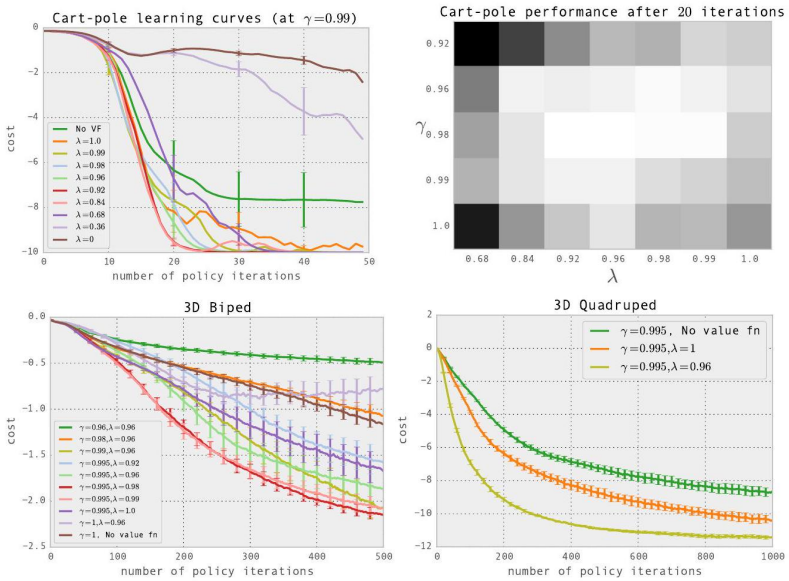
**Value fit with a trust region**

$$\min_\phi \sum_n \left\| V_\phi(s_n) - \hat{V}_n \right\|^2 \quad \text{s.t.} \quad \frac{1}{N} \sum_n \frac{\|V_\phi(s_n) - V_{\phi_{\text{old}}}(s_n)\|^2}{2\sigma^2} \le \epsilon$$

- $=$ a KL trust region for a Gaussian with mean $V_\phi(s)$; $\sigma^2$ is the current mean squared error.
- Solved with conjugate gradient, like TRPO.
- Keeps $V$ from overfitting the latest batch — the "stable improvement" challenge of the abstract.

Unlike Q-learning-style critics (e.g. SAC, which learns $Q(s,a)$ off-policy), GAE's critic is a state value $V(s)$ fitted **on-policy**. Networks: three hidden layers (100, 50, 25 tanh units) for both policy and value. TD($\lambda$) targets for the critic made no difference.

# 26 Results: choosing $\gamma$ and $\lambda$




GAE Figs. 2–3. Cost = negative reward (lower is better).

**Cart-pole** (top)

- best at intermediate $\lambda \in [0.92, 0.99]$, $\gamma \in [0.96, 0.99]$
- $\lambda = 0$ and "No VF" (time-dependent baseline only) much worse

**3D biped, quadruped** (bottom)

- best $\gamma \in [0.99, 0.995]$, $\lambda \in [0.96, 0.99]$
- the biped runs after 1000 iterations $\approx$ 5.8 days of simulated time

**Key idea**

Best $\lambda <$ best $\gamma$: $\lambda$ adds bias only if $V$ is wrong. Discussion: $\lambda \in [0.9, 0.99]$ usually best; App. A.2: the $\lambda = 0$ estimator was "prohibitively" biased.

Part IV

# Proximal Policy Optimization

**The question of Part IV:** TRPO's second-order step uses each batch **once**. PPO wants **many first-order SGD steps** per batch without leaving the trust region. It **clips** the importance ratio, so the objective itself stops rewarding large moves.

## 27 Setting and motivation

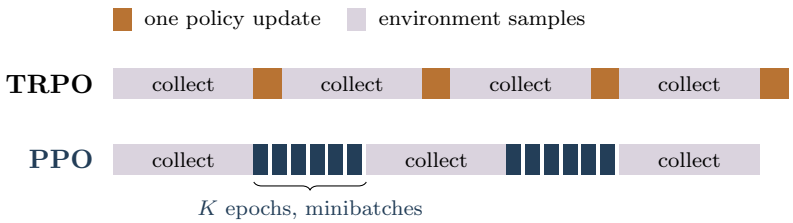
**Proximal Policy Optimization Algorithms**
Schulman, Wolski, Dhariwal, Radford & Klimov · arXiv 2017 (OpenAI)

**Introduces:** PPO — TRPO's reliability with only first-order SGD
Local copy: `proximal_policy_optimization_algorithms.md`

- Same setting as TRPO; advantages $\hat{A}_t$ from (truncated) **GAE**.
- **TRPO**: complicated; no dropout, no parameter sharing between policy and value.
- **Vanilla PG**: several steps on the same batch ⇒ destructively large updates.
- **Goal**: reuse each batch for many SGD steps, without leaving the trust region.
- Tested on MuJoCo and 49 Atari games.

"Vanilla" PG estimator: $\hat{g} = \hat{\mathbb{E}}_t[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t]$, from the loss $L^{PG} = \hat{\mathbb{E}}_t[\log \pi_\theta \hat{A}_t]$; $\hat{\mathbb{E}}_t$ is the empirical average over a batch. Q-learning with function approximation is also dismissed: it fails on many simple problems and had no strong continuous-control results at the time.

### 27.1 The goal: many SGD steps per batch




**TRPO**

One (expensive, carefully constrained) step, then the batch is thrown away.

**PPO**

Squeeze the same batch with cheap first-order steps: $K$ epochs of minibatch SGD.

**Why naive reuse fails**

The data came from $\pi_{\text{old}}$: the surrogate is trustworthy only near $r_t = 1$. For one sample with $\hat{A}_t > 0$ the term $r_t\hat{A}_t$ is linear in $r_t$ — **nothing says stop**. Unchecked, repeated steps give **destructively large** updates. Only TRPO's KL constraint stopped it; PPO asks how to stop it with an ordinary first-order optimizer.

## 28 Back to the CPI surrogate

The ratio is TRPO's importance weight:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \qquad r_t(\theta_{\text{old}}) = 1.$$

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t\big[r_t(\theta)\,\hat{A}_t\big] \qquad (\text{“CPI”: Kakade \& Langford, where this objective was proposed})$$

It is TRPO's $L_{\theta_{\text{old}}}$ written with sample averages; its gradient at $\theta_{\text{old}}$ is the vanilla policy gradient. Many SGD steps on $L^{CPI}$ alone: $r_t$ runs far from 1 $\Rightarrow$ huge updates. Normalized score $-0.39$: worse than random (§32.1).

**Three ways to stay close:**

**Constraint (TRPO)**

s.t. $\hat{\mathbb{E}}_t[D_{\text{KL}}] \le \delta$

second-order solver

**Penalty (baseline)**

$-\beta\,\hat{\mathbb{E}}_t[D_{\text{KL}}]$

no single $\beta$ works $\rightarrow$ adapt it

**Clip (“PPO”)**

no gain once $|r_t - 1| > \epsilon$

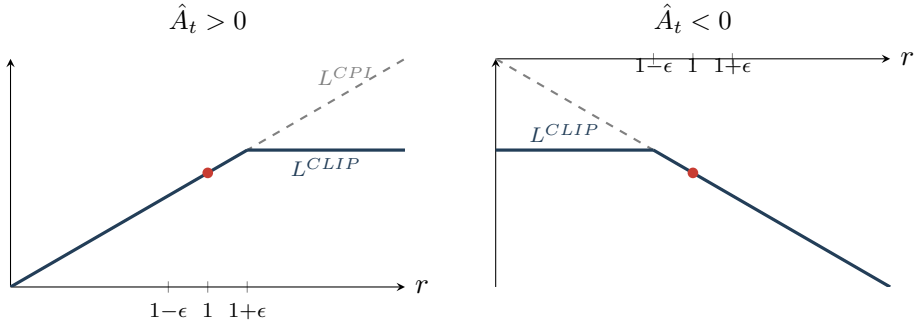
first order

The penalty form is what TRPO's theory suggests; TRPO used a constraint because “it is hard to choose a single value of $\beta$ that performs well across different problems — or even within a single problem”.

## 29 The clipped surrogate objective

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big], \qquad \epsilon = 0.2$$

One term of the sum, as a function of the ratio $r$:




Redrawn from PPO Fig. 1. Red dot: the start of the optimization, $r = 1$. Dashed: $L^{CPI}$.

**$\hat{A}_t > 0$: raise $\pi(a_t \mid s_t)$**

- No extra gain past $1+\epsilon$ (flat).
- Wrong way ($r < 1-\epsilon$): **not** clipped, still costs.

**$\hat{A}_t < 0$: lower $\pi(a_t \mid s_t)$**

- No extra gain below $1-\epsilon$ (flat).
- Wrong way ($r > 1+\epsilon$): **not** clipped, the loss keeps growing.

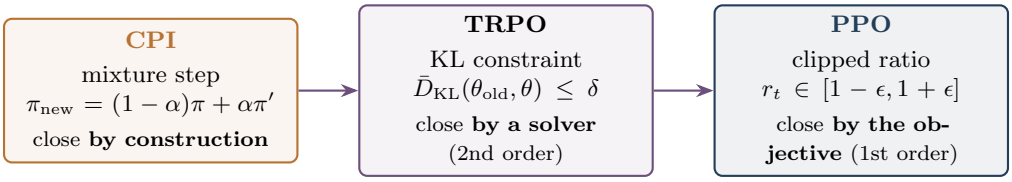
**A pessimistic bound**

1. The min gives $L^{CLIP} \le L^{CPI}$: clipping removes **gains**, never hides **losses**.
2. Moving the wrong way keeps its gradient: the update can always pull $r_t$ **back** toward 1.

3. $L^{CLIP} = L^{CPI}$ to first order at $\theta_{\text{old}}$.

PPO Fig. 2: along the update direction, $L^{CLIP}$ peaks at a KL of about 0.02 (Hopper).

### 29.1 What clipping replaces




**Proximal** = keep $\pi_\theta$ near $\pi_{\text{old}}$ — with plain minibatch SGD/Adam, several epochs per batch.

**Clipping is an incentive, not a limit**

Clipping removes the **incentive** to move a ratio further — it does not **enforce** a limit. Clipped samples can still drift through shared parameters, moved by the gradients of unclipped samples, so **no KL bound is guaranteed**. In practice the KL stays small; some implementations add early stopping on the KL (cf. Engstrom et al., 2020, "Implementation Matters in Deep Policy Gradients").

## 30 PPO in practice

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\Big[\underbrace{L_t^{CLIP}(\theta)}_{\text{policy}} - c_1 \underbrace{\big(V_\theta(s_t) - V_t^{\text{targ}}\big)^2}_{\text{value}} + c_2 \underbrace{S[\pi_\theta](s_t)}_{\text{entropy}}\Big]$$

1. $c_1$ matters only when policy and value **share parameters** (one network, one loss).
2. The entropy bonus $S$ is PPO's only explicit exploration term ("to ensure sufficient exploration") — it acts on the **action** distribution, not on where episodes start.

**Algorithm: PPO, actor–critic style (Alg. 1)**

1. $N$ actors run $\pi_{\theta_{\text{old}}}$ for $T$ steps each.
2. Truncated GAE: $\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}$.
3. $K$ epochs of minibatch Adam on $L$.
4. $\theta_{\text{old}} \leftarrow \theta$.

| | MuJoCo | Atari |
|---|---|---|
| horizon $T$ | 2048 | 128 |
| epochs $K$ | 10 | 3 |
| minibatch | 64 | $32 \times 8$ |
| actors $N$ | — | 8 |
| Adam step | $3 \cdot 10^{-4}$ | $2.5 \cdot 10^{-4}\alpha$ |
| $\gamma$, $\lambda$ | 0.99, 0.95 | 0.99, 0.95 |
| clip $\epsilon$ | 0.2 | $0.1\alpha$ |

PPO Tables 3, 5. Atari: $\alpha$ annealed $1 \to 0$, $c_1 = 1$, $c_2 = 0.01$.

$K$ epochs on the same batch is the point: TRPO took **one** step per batch. Truncated GAE does not look beyond the segment end $T$, which suits fixed-length segments and recurrent nets.

# 31 TRPO vs. PPO

| | TRPO | PPO |
|---|---|---|
| stay close by | KL constraint $\bar{D}_{\mathrm{KL}} \le \delta$ | clipped ratio $[1-\epsilon, 1+\epsilon]$ |
| solver | CG + line search (2nd order) | minibatch Adam (1st order) |
| steps per batch | one | $K$ epochs |
| policy + value | separate networks | may share (one loss) |
| advantages | Monte Carlo $\hat{Q}$ | truncated GAE |
| states from | $s_0 \sim \rho_0$, then $\pi_{\mathrm{old}}$ | same — still no $\mu$ |

**Kept from CPI and TRPO**

- Surrogate on the **old** samples
- The ratio $\pi_\theta / \pi_{\theta_{\mathrm{old}}}$
- Small, controlled steps

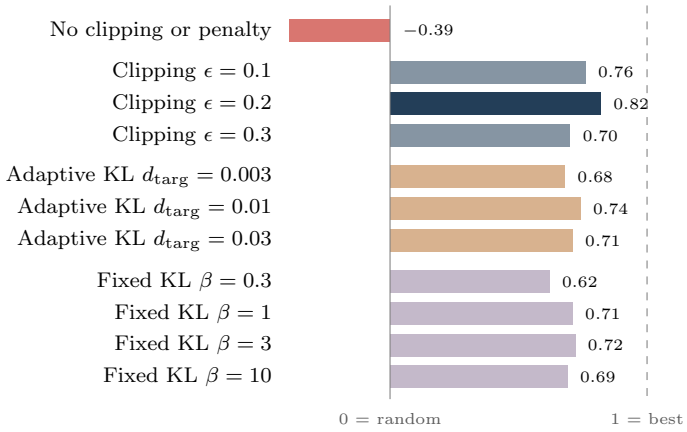
**Given up**

- Any improvement **bound**
- An exact limit on the KL
- (Monotone improvement: never had it in practice)

TRPO's practical algorithm already dropped the guarantee (mean KL, fixed $\delta$, estimated advantages); PPO drops the remaining machinery and keeps the heuristic.

# 32 Results

## 32.1 Clipping vs. KL penalty: the ablation




PPO Table 1: 7 MuJoCo tasks × 3 seeds, 1M steps.

**Adaptive KL penalty**

$$L^{KLPEN} = \hat{\mathbb{E}}_t\left[r_t \hat{A}_t - \beta\, D_{\mathrm{KL}}[\pi_{\theta_{\mathrm{old}}}, \pi_\theta]\right]$$

After each update, with $d = \hat{\mathbb{E}}_t[D_{\mathrm{KL}}]$:

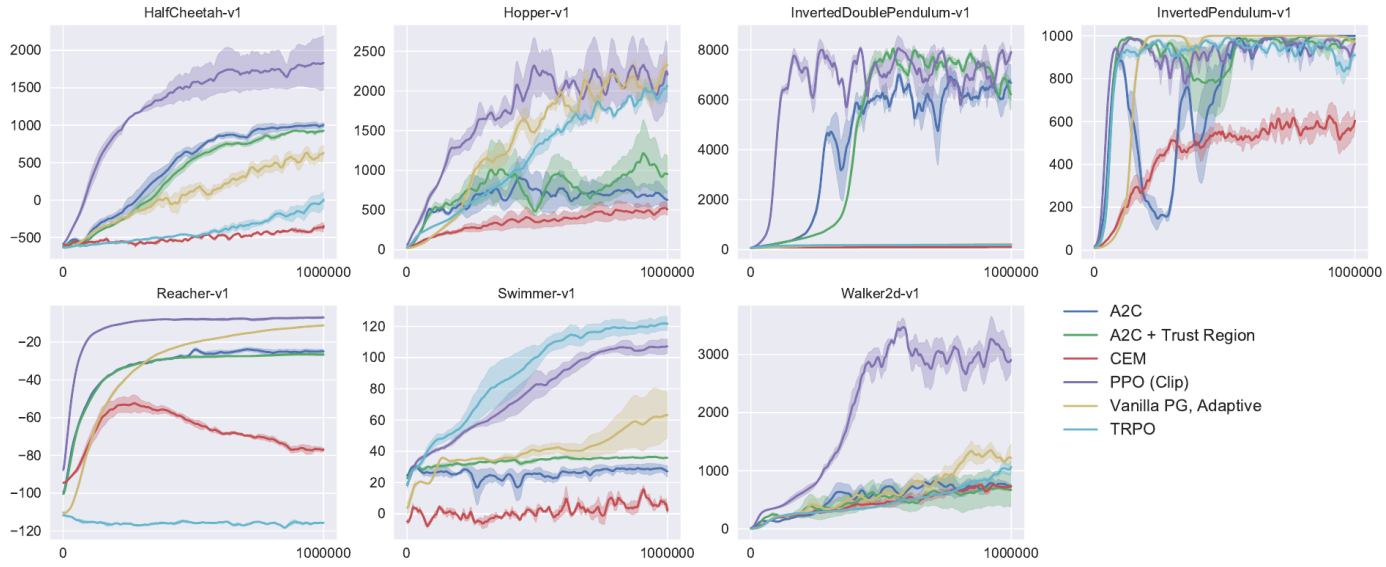
- $d < d_{\mathrm{targ}}/1.5$: $\beta \leftarrow \beta/2$
- $d > 1.5\, d_{\mathrm{targ}}$: $\beta \leftarrow 2\beta$

Constants 1.5 and 2 are heuristic; not sensitive to them.

**Key idea**

Clipping with $\epsilon = 0.2$ wins; the KL penalty stays a useful baseline.

## 32.2 PPO vs. other algorithms




PPO Fig. 3: MuJoCo, one million timesteps.

- **MuJoCo**: **PPO (clip)** best on almost all 7 tasks — above TRPO, A2C, CEM, vanilla PG. Swimmer is the one task where TRPO is clearly ahead.
- Also (not shown): 3D humanoid running, steering and getting up.

**Atari**, 49 games — games won

| | A2C | ACER | **PPO** |
|---|---|---|---|
| avg. over training | 1 | 18 | **30** |
| last 100 episodes | 1 | **28** | 19 |

"avg. over training" favors fast learning.

# 33 The whole story: CPI → TRPO → GAE → PPO

| | **CPI** | **TRPO** | **GAE** | **PPO** |
|---|---|---|---|---|
| **A. how far may I step?** | mixture step $\alpha$ | KL trust region $\delta$ | trust region for $\pi$ **and** $V$ | clipped ratio $\epsilon$ |
| **how is it solved?** | greedy chooser + formula for $\alpha$ | CG + line search | CG (policy and value) | Adam, $K$ epochs |
| **advantage estimate** | rollouts from restarts | Monte Carlo $\hat{Q}$ | GAE$(\gamma, \lambda)$ + critic | truncated GAE |
| **B. where do states come from?** | restarts $s_0 \sim \mu$ | $\rho_0$ (vine: resets to visited states) | $\rho_0$ | $\rho_0$ (+ entropy bonus on actions) |

1. **Row A**: the same idea at every stage — keep the new policy close to the one that produced the data, because the surrogate uses the old state distribution. By construction (CPI), by a constraint (TRPO), by the objective (PPO).
2. **Rows 2–3**: the price of keeping close falls, from a theoretical step size to an off-the-shelf optimizer, and from restart rollouts to a learned critic.
3. **Row B**: CPI's second fix is **not** carried forward.

### 33.1 One line per paper

| Paper | Problem it attacks | Main idea |
|---|---|---|
| **CPI** | greedy updates from approximate values can hurt | conservative mixture, step $\alpha$ from a bound |
| **CPI** | on-policy samples miss important states | exploratory restart distribution $\mu$ |
| **TRPO** | mixtures don't fit neural-net policies | step size = a policy distance (KL) |
| **TRPO** (practice) | the theoretical penalty is far too conservative | mean-KL trust region, CG + line search |
| **GAE** | $\hat{A}_t$ from returns is too noisy | critic $V$ + bias–variance knob $\lambda$ |
| **PPO** | second-order step, one step per batch | clipped ratio + minibatch SGD, $K$ epochs |

**Take-home**

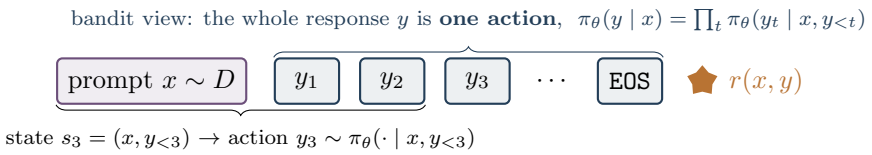
One idea — **stay close to the data-generating policy, measured on its own states** — runs from CPI to PPO, made cheaper at each step. CPI's state-coverage idea was dropped along the way.

# Part V

# RL for Language Models

**The question of Part V:** what does the CPI → PPO machinery look like when the "environment" is text and the reward is a learned model of human preferences? The part is organized around one question — **what can we remove?** — from InstructGPT's four models down to plain REINFORCE.

## 34 Text generation as an RL problem




**Start distribution $D$**

The prompts users send — "$D$ comes from the application" (Part I).

**Dynamics**

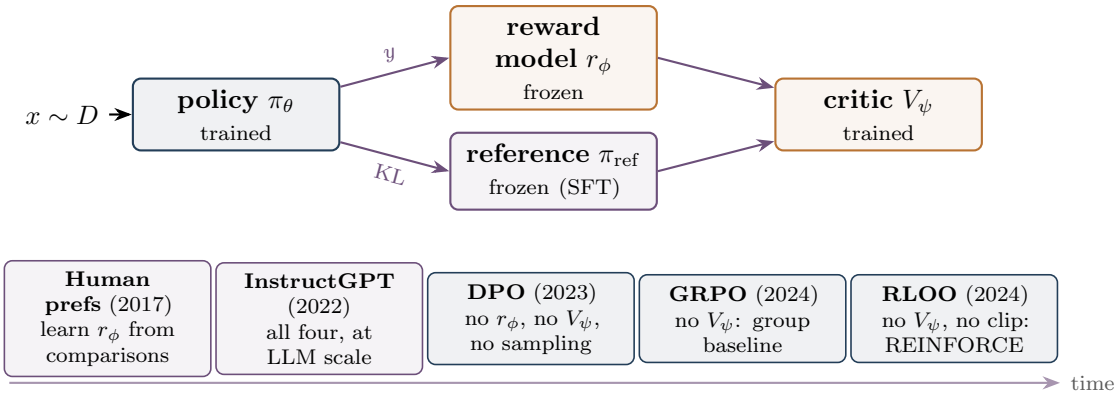
Append the token: **known** and **deterministic**.

**Reward**

Only at the end — but who writes $r(x, y)$ for "helpful"?

- Token-level MDP as in RLHF with PPO: discount 1, reward-model score at `EOS`, per-token KL elsewhere (RLOO paper, Sec. 2.1).
- InstructGPT calls it "a bandit environment which presents a random customer prompt and expects a response" (Sec. 3.5).
- Our remark: deterministic, known dynamics make resetting to any prefix free — exactly what CPI's restarts $\mu$ and TRPO's vine needed. The open question is the **reward**.

### 34.1 RLHF with PPO: what can we remove?




PPO for RLHF "typically requires loading up to 4 models simultaneously: the generator, the reference (for KL estimation), the critic, and the reward model" (RLOO paper, Sec. 1). In InstructGPT the critic is initialized from the reward model (6B each).

# 35 Learning the reward from human preferences

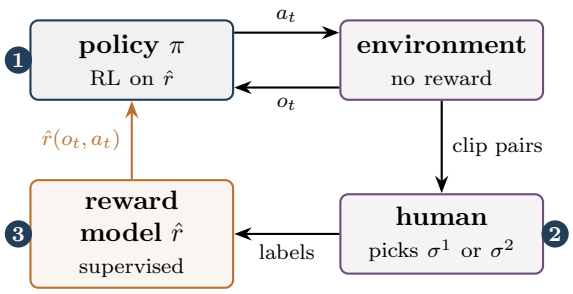
**Deep Reinforcement Learning from Human Preferences**
Christiano, Leike, Brown, Martic, Legg & Amodei · NeurIPS 2017

**Introduces:** a reward model learned from pairwise human comparisons
Local copy: `rlhf.md`

1. Observations $o_t$, actions $a_t$ (Atari, MuJoCo) — **no reward** from the environment.
2. A human compares two short clips $\sigma^1, \sigma^2$ (1–2 s): which is better?
3. Goal: behavior the human prefers, with **few** queries.
4. No resets to chosen states: clips start wherever the agent was.
5. The policy is trained by any RL method: **TRPO** (MuJoCo), **A2C** (Atari).

Motivation: tasks we can recognize but not demonstrate or specify (a backflip). Comparisons were easier for humans than absolute scores.

## 35.1 Three processes, running asynchronously




1. RL maximizes the **predicted** reward $\sum_t \hat{r}(o_t, a_t)$.
2. Pairs of clips go to the human.
3. $\hat{r}$ is fit to all comparisons so far.

**Few labels**

Feedback on $< 1\%$ of interactions. MuJoCo: 700 labels $\approx$ RL with the true reward. Backflip: 900 queries, $< 1$ hour.

**Keep feedback in the loop**

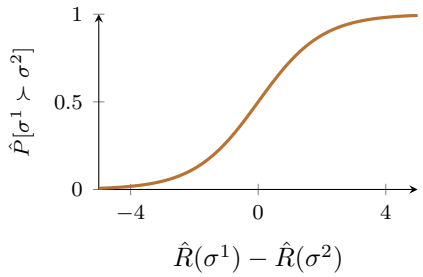
Training $\hat{r}$ only on early queries fails: the policy **exploits** the partial reward (e.g. endless volleys in Pong).

Queries are chosen where an ensemble of reward predictors disagrees most. Rewards from $\hat{r}$ are normalized to zero mean (only differences are identified).

## 35.2 Fitting the reward: the Bradley–Terry model

Preference = logistic in the **difference** of summed rewards:

$$\hat{P}[\sigma^1 \succ \sigma^2] = \frac{e^{\sum_t \hat{r}(o_t^1, a_t^1)}}{e^{\sum_t \hat{r}(o_t^1, a_t^1)} + e^{\sum_t \hat{r}(o_t^2, a_t^2)}} = \sigma\big(\hat{R}(\sigma^1) - \hat{R}(\sigma^2)\big)$$




Like **Elo** in chess: the rating gap predicts who wins.

Fit $\hat{r}$ by cross-entropy on the labels $\mu$ (all mass on the preferred clip, uniform if "equally good"):

$$\text{loss}(\hat{r}) = -\sum_{\mathcal{D}} \Big[\mu(1)\log \hat{P}[\sigma^1 \succ \sigma^2] + \mu(2)\log \hat{P}[\sigma^2 \succ \sigma^1]\Big]$$

**Only differences matter**

$\hat{r} + c$ fits equally well $\Rightarrow$ normalize. **This invariance is what DPO exploits.** For LLMs the clips become responses $y$ to the same prompt $x$.

# 36 InstructGPT: RLHF at LLM scale

**Training Language Models to Follow Instructions with Human Feedback**
Ouyang, Wu, Jiang, et al. (OpenAI) · NeurIPS 2022

**Introduces:** InstructGPT — the SFT → reward model → PPO recipe
Local copy: `2203.02155v1.pdf`

1. $D$: real prompts sent to the OpenAI API, plus labeler-written ones.
2. **Bandit** environment: a prompt, a response, a reward from the RM, end of episode.
3. Rewards from a 6B **reward model** trained on labeler rankings.
4. Per-token **KL penalty** to the SFT model.
5. GPT-3 architecture: 1.3B, 6B, 175B.

## 36.1 The three-step recipe

**1. SFT**

- Labelers **write** answers.
- Supervised fine-tuning of GPT-3.
- 13k prompts

**2. Reward model**

- Labelers **rank** $K = 4$–$9$ outputs → $\binom{K}{2}$ pairs.
- Loss $-\log \sigma\big(r_\theta(x, y_w) - r_\theta(x, y_l)\big)$.
- 33k prompts; 6B RM

**3. PPO**

- New prompt → the policy writes $y$.
- RM scores $y$ → PPO step (critic initialized from the RM).
- 31k prompts, API only

**Definition: reward-model loss (InstructGPT Eq. 1)**

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}}\mathbb{E}_{(x,y_w,y_l)\sim D}\Big[\log \sigma\big(r_\theta(x, y_w) - r_\theta(x, y_l)\big)\Big]$$

The Bradley–Terry loss of the previous section, with responses instead of clips.

- **All $\binom{K}{2}$ pairs in one batch element**: treating them as separate data points made each completion appear in $K - 1$ updates, and the RM overfit after one epoch.
- The RM is the SFT model with the unembedding layer replaced by a scalar head; 175B RMs were unstable, so all RMs are 6B.
- SFT: 16 epochs — overfits validation loss after 1 epoch, but more epochs help RM score and human ratings.

### 36.2 The KL-regularized objective (PPO-ptx)

$$\text{objective}(\phi) = \mathbb{E}_{(x,y)\sim D_{\pi_\phi^{RL}}}\Big[\underbrace{r_\theta(x,y)}_{\text{reward model}} - \underbrace{\beta \log \frac{\pi_\phi^{RL}(y \mid x)}{\pi^{SFT}(y \mid x)}}_{\text{stay near SFT}}\Big] + \underbrace{\gamma\, \mathbb{E}_{x\sim D_{\text{pre}}}\big[\log \pi_\phi^{RL}(x)\big]}_{\text{pretraining mix}}$$

**Reward**

The RM is only a **proxy** for the labelers.

**KL, $\beta = 0.02$**

Per token, against **over-optimizing** the RM. Its expectation is $D_{\text{KL}}(\pi^{RL} \| \pi^{SFT})$.

**ptx, $\gamma > 0$**

Fixes the "alignment tax" on public NLP benchmarks (SQuAD, DROP, HellaSwag, WMT).

**Results**

Labelers prefer the **1.3B** InstructGPT to the **175B** GPT-3. 175B vs. 175B: preferred $85 \pm 3\%$ of the time. TruthfulQA about twice GPT-3; closed-domain hallucination 21% vs. 41%.

> Without ptx ($\gamma = 0$) this is the RLHF objective that **DPO** starts from. Here $\gamma$ is a mixing weight, not a discount. "PPO" models: $\gamma = 0$; "PPO-ptx": $\gamma > 0$. The paper aligns GPT-3 "to the stated preferences of a specific group of people (mostly our labelers and researchers)", not to broader human values (Sec. 5.2).

## 37 DPO: the RLHF objective as a classification loss

**Direct Preference Optimization: Your Language Model Is Secretly a Reward Model**
Rafailov, Sharma, Mitchell, Ermon, Manning & Finn · NeurIPS 2023

**Introduces:** DPO — the RLHF objective solved with a classification loss
Local copy: `dpo/dpo.md`

1. Same objective as RLHF: reward minus $\beta\, D_{\text{KL}}(\pi \,\|\, \pi_{\text{ref}})$, with $\pi_{\text{ref}} = \pi^{SFT}$.
2. Preferences follow **Bradley–Terry** with a latent reward $r^*$.
3. A **fixed** dataset $\mathcal{D} = \{(x, y_w, y_l)\}$ (offline).
4. No reward model, **no sampling** from $\pi_\theta$ during training, no critic.

### 37.1 From the RLHF objective to a loss on the policy

Four steps.

1. **The RLHF objective**, for a reward $r$:

$$\max_\pi \; \mathbb{E}_{x,\,y\sim\pi}\big[r(x,y)\big] - \beta\, D_{\text{KL}}\big[\pi \,\|\, \pi_{\text{ref}}\big]$$

2. **Its optimum in closed form** — tilt $\pi_{\text{ref}}$ by $e^{r/\beta}$:

$$\pi_r(y \mid x) = \frac{1}{Z(x)}\, \pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}, \qquad Z(x) = \textstyle\sum_y \pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}$$

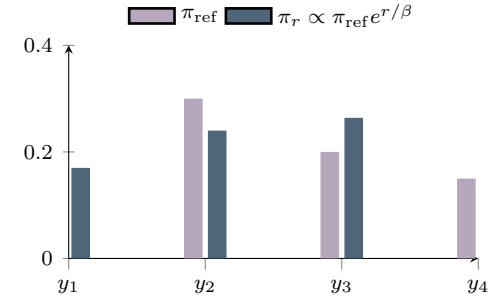
Why: the objective equals $-\beta\, D_{\text{KL}}(\pi \| \pi_r) + \beta \log Z(x)$, minimized at $\pi = \pi_r$ (DPO App. A.1). $Z(x)$ is intractable.

3. **Solve for the reward**:

$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

4. **Into Bradley–Terry**: both responses share $x$, so $\beta \log Z(x)$ **cancels**:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{\mathcal{D}}\Big[\log \sigma\big(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)\big)\Big], \qquad \hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$




Toy example: $\beta = 1$, rewards $r = 0, 0.5, 1, 1.5$. High-reward answers gain mass.

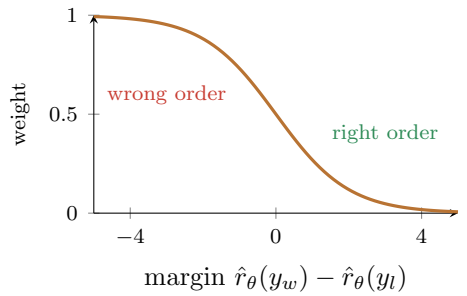
**Read it backwards**

**Every policy defines a reward**, $\hat{r}_\theta = \beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$, up to a term that depends on $x$ only. The reward-model loss with $r_\phi$ replaced by $\hat{r}_\theta$: **the LM is the reward model**.

Thm. 1 of DPO: the reparameterization loses no generality — every reward class (rewards equal up to a function of $x$) contains one of the form $\beta \log \pi/\pi_{\text{ref}}$. Link to Part I: the optimum is a *soft-greedy* policy, and $Z(x)$ plays the role of a soft value of $\pi_{\text{ref}}$. DPO also works with Plackett–Luce rankings (App. A.3), i.e. InstructGPT-style $K$-way rankings.

## 37.2 What the DPO update does

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \mathbb{E}_{\mathcal{D}}\Big[\underbrace{\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big)}_{\text{large when the ranking is wrong}} \big(\underbrace{\nabla_\theta \log \pi_\theta(y_w \mid x)}_{\text{raise } y_w} - \underbrace{\nabla_\theta \log \pi_\theta(y_l \mid x)}_{\text{lower } y_l}\big)\Big]$$




- Like the reward-model loss — applied to the policy itself.
- The weight **fades** once a pair is ranked right. Without it (unlikelihood: equal weights) the model **degenerates**.
- **Sentiment** (IMDb): best reward for every KL budget, above PPO — even PPO with the true reward.
- **TL;DR**: 61% win rate vs. PPO's 57%; more robust to sampling temperature.

## 37.3 What we removed, and what it costs

**Removed**

- The reward model — implicit in $\pi_\theta$.
- The critic and the RL loop.
- Sampling during training.

Left: $\pi_\theta$, $\pi_{\text{ref}}$ and one supervised-style loss. Same optimum as RLHF (under Bradley–Terry).

**Costs**

- **Offline**: responses come from $\pi^{SFT}$, not $\pi_\theta$ — no exploration.
- Learns only on the pairs in $\mathcal{D}$.
- Later evidence: online beats offline (DeepSeek-Math); DPO answers are long (RLOO paper: 104 tokens vs. 60.6 for RLOO on HH).

**Part I echo:** CPI's quality depends on how well the sampling distribution covers the good states ($\|d_{\pi^*}/\mu\|_\infty$, Thm. 6.2). Here: how well $\pi^{SFT}$'s answers cover good answers. (Our analogy; the DPO paper itself does not state these costs in this form.)

# 38 GRPO: PPO without the critic

**DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models**

Shao, Wang, Zhu, et al. (DeepSeek-AI) · arXiv 2024

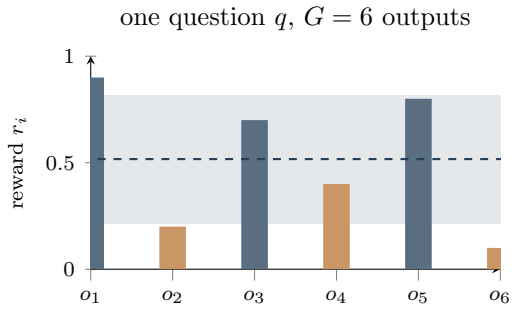
**Introduces:** GRPO — PPO with a group baseline instead of a critic

Local copy: `deepseekmat_pushing_the_limits_of_....md`

1. Math questions $q$ (GSM8K, MATH; chain of thought); policy: DeepSeekMath-Instruct 7B.
2. A **reward model** scores each output $o$ — at the end (outcome) or per step (process).
3. Online: $G = 64$ outputs per question from the current policy.
4. The problem with PPO's critic: **as large as the policy**, and hard to fit per token when the reward comes only at the end.

Most of the paper is about pre-training data (120B math tokens); GRPO is Sec. 4. KL coefficient 0.04, learning rate $10^{-6}$, batch 1024. GRPO later became the RL algorithm of DeepSeek-R1.

## 38.1 Group-relative advantages




Toy rewards. Blue: above the mean (pushed up). Copper: below (pushed down).

Sample $G$ outputs for the **same** question and score them:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

The same value for every token $t$ of $o_i$ (outcome supervision).

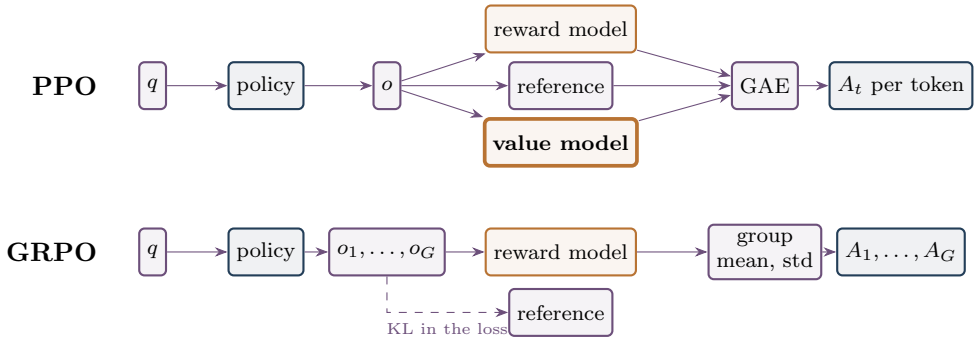
- The group mean replaces $V(q)$: a critic computed **on the fly**.
- Matches how RMs are trained: **comparisons** within one question.

**Results**

RL on GSM8K + MATH questions only: GSM8K 82.9 → 88.2%, MATH 46.8 → 51.7%, and gains out of domain (CMATH 84.6 → 88.8%).

**Process supervision** (Sec. 4.1.3): normalize per-step rewards across the group; token $t$ gets the sum of normalized rewards of the steps ending at or after $t$. It did better than outcome supervision. Sec. 5.2.2: RL improves Maj@K but not Pass@K — it makes the output distribution more robust rather than adding new abilities.

### 38.2 PPO vs. GRPO




$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\Big\{\min\big(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1\pm\varepsilon)\,\hat{A}_{i,t}\big) - \beta\, D_{\text{KL}}\big[\pi_\theta \| \pi_{\text{ref}}\big]\Big\}\Big]$$

$\rho_{i,t} = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$: still PPO's clipped ratio, per token.

1. **Critic gone**: the group statistics replace $V_\psi$ and GAE.
2. **KL moved**: PPO for LLMs adds the per-token KL to the reward, so it passes through GAE; GRPO puts it in the loss, which keeps $\hat{A}$ simple. KL estimator ("k3"): $\frac{\pi_{\text{ref}}}{\pi_\theta} - \log\frac{\pi_{\text{ref}}}{\pi_\theta} - 1 \geq 0$, unbiased under samples from $\pi_\theta$.

**The clip never acts in their runs**

"The policy model only has a single update following each exploration stage" (Sec. 4.2), so $\pi_\theta = \pi_{\theta_{\text{old}}}$ when the gradient is taken: the ratio is 1 and the clip is inactive — the gradient is **REINFORCE with a group baseline**. Iterative GRPO (Alg. 1) also retrains the RM and resets $\pi_{\text{ref}} \leftarrow \pi_\theta$ between rounds.

## 39 RLOO: back to REINFORCE

**Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs**

Ahmadian, Cremer, Gallé, et al. (Cohere) · ACL 2024

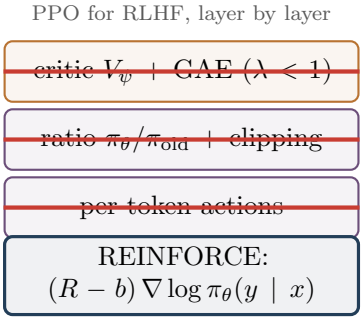
**Introduces:** REINFORCE with a leave-one-out baseline beats PPO for RLHF

Local copy: `REINFORCE_leave_1_out/`

1. Standard RLHF: SFT → reward model $r_\phi$ → online RL.
2. KL-shaped reward per response: $R(x, y) = r_\phi(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$.
3. **Claim**: PPO's tools fight high variance from a **weak** start. A pretrained LLM is a **strong** start.
4. Reward only at `EOS`, deterministic dynamics ⇒ treat $y$ as **one action**.

"Strong initialization coupled with prompt conditioning leads to the concentration of probability mass on a few tokens at each generation step" (Sec. 3.1). Models: Pythia 6.9B, Llama-7B; data: TL;DR and Anthropic-HH.

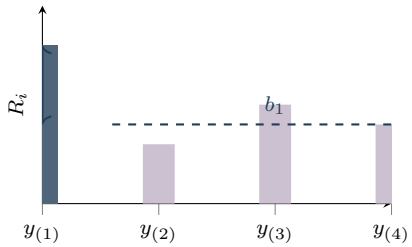
## 39.1 Which parts of PPO does RLHF need?




1. **Bias for variance? Not worth it.** Reward falls steadily as $\lambda$ goes $1 \to 0$; $\lambda = 1$ (no bootstrapping) is best.
2. **Clipping? Rarely active.** Clipped in $< 5\%$ of each batch; removing ratio and clip does not hurt ("if anything a slight boost").
3. **Tokens as actions? Not needed.** The whole response as one action works as well or better.

> Callback to Parts III–IV: these are exactly the tools GAE and PPO added for robot control, where the start is a random policy. REINFORCE with a baseline vs. vanilla PG (per token): 70.7 vs. 70.4 (TL;DR), 37.9 vs. 36.4 (HH-Pythia), 55.3 vs. 52.3 (HH-Llama).

## 39.2 The leave-one-out baseline




$b_1$ = mean of the **other** samples.

$k$ samples $y_{(i)} \sim \pi_\theta(\cdot \mid x)$, rewards $R_i = R(y_{(i)}, x)$:

$$\frac{1}{k}\sum_{i=1}^{k}\Big[R_i - \underbrace{\frac{1}{k-1}\sum_{j\neq i} R_j}_{b_i}\Big]\nabla \log \pi_\theta(y_{(i)} \mid x)$$

The baseline does not depend on $y_{(i)} \Rightarrow$ **unbiased**: $\mathbb{E}[b_i \nabla \log \pi(y_{(i)})] = 0$.

**GRPO's mean vs. leave-one-out**

GRPO's mean includes $y_{(i)}$ itself, but

$$R_i - \bar{R} = \tfrac{k-1}{k}\big(R_i - b_i\big),$$

the same up to scale. The real differences are the division by the **std** and the $1/|o_i|$ token average — they reweight harder/easier questions and longer/shorter answers (cf. Liu et al., 2025, "Understanding R1-Zero-Like Training").

**Results**

RLOO ($k = 4$) beats PPO's win rate by 10.3 (TL;DR), 14.5 (HH, Pythia) and 32.1 (HH, Llama) points — with **three models, no critic**. It also beats RAFT (fine-tune on the best of $k$) at the same sampling budget, and DPO.

# 40 Summary of Part V

## 40.1 What did we remove?

| | Reward model | Critic | KL to $\pi_{\text{ref}}$ | Samples of $\pi_\theta$ | Clip |
|---|---|---|---|---|---|
| InstructGPT (PPO) | ✓ | ✓ | ✓ in reward | ✓ | ✓ |
| DPO | implicit $\hat{r}_\theta$ | × | ✓ via $\beta$ | × offline | × |
| GRPO | ✓ | × group | ✓ in loss | ✓ $G$ per $q$ | ✓ |
| RLOO | ✓ | × leave-one-out | ✓ in reward | ✓ $k$ per $x$ | × |

**Never removed: the KL to $\pi_{\text{ref}}$**

The learned reward is only trusted near $\pi^{SFT}$: InstructGPT adds the KL "to mitigate over-optimization of the reward model"; DPO: it keeps the policy "from deviating too far from the distribution on which the reward model is accurate".

**Why the critic could go**

Many samples per prompt are cheap, and the prompt is the only real state.

## 40.2 The thread: from CPI to LLMs

| | CPI | TRPO | PPO / GAE | LLMs |
|---|---|---|---|---|
| **surrogate on old samples** | policy advantage $\mathbb{A}_{\pi,\mu}$ | $L_{\theta_{\text{old}}}$ + importance weights | $L^{CPI} = \hat{\mathbb{E}}[r_t \hat{A}_t]$ | per-token ratio (GRPO) or plain REINFORCE |
| **stay close** | mixture step $\alpha$ | KL trust region $\delta$ | clip $\epsilon$ | KL to $\pi_{\text{ref}}$ ($\beta$): **a new reason** |
| **advantage / baseline** | $A_\pi$ from restarts | Monte Carlo $\hat{Q}$ | GAE$(\gamma, \lambda)$ with a critic | group mean / leave-one-out |
| **where states come from** | $D$, restarts $\mu$ | $\rho_0$ (vine: resets) | $\rho_0$ | prompts $x \sim D$ |

1. **Row 1**: PPO names its surrogate after CPI.
2. **Row 2**: the *reason* changes. In CPI/TRPO/PPO the step is small so the surrogate stays accurate; in RLHF the KL to $\pi_{\text{ref}}$ keeps the policy where the learned reward model is accurate. The old/new-policy closeness is often barely active (RLOO: clipped $< 5\%$).
3. **Row 3**: the same bias–variance question, answered with less and less machinery as samples per prompt get cheap.
4. **Row 4**: the start distribution $D$ of Part I is the prompt distribution; deterministic, known dynamics would even allow CPI-style restarts.

# Appendix

Proofs, tools and side comparisons. Each appendix is self-contained and is referenced from the main text.

## A Proof: the policy advantage is the derivative at $\alpha = 0$

**Claim** (K&L Sec. 4.1, Eq. 4.2): $\left.\frac{\partial \eta_\mu}{\partial \alpha}\right|_{\alpha=0} = \frac{1}{1-\gamma}\mathbb{A}_{\pi,\mu}(\pi')$.

**Step 1 — the mixtures are a one-parameter policy class.**

$$\pi_\alpha(a;s) = (1-\alpha)\,\pi(a;s) + \alpha\,\pi'(a;s) \quad \Longrightarrow \quad \frac{\partial \pi_\alpha(a;s)}{\partial \alpha} = \pi'(a;s) - \pi(a;s)$$

**Step 2 — the policy gradient theorem holds for *any* parameter, so take $\theta := \alpha$.**

**Policy gradient, any $\theta \in \mathbb{R}^m$**

$$\nabla_\theta \eta_\mu = \frac{1}{1-\gamma}\sum_s d_{\pi_\theta,\mu}(s)\sum_a \nabla_\theta \pi_\theta(a;s)\,Q_{\pi_\theta}(s,a)$$

**Mixture, at $\alpha = 0$ ($\pi_0 = \pi$)**

$$\left.\frac{\partial \eta_\mu}{\partial \alpha}\right|_0 = \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d_{\pi,\mu}}\Big[\textstyle\sum_a (\pi' - \pi)(a;s)\,Q_\pi(s,a)\Big]$$

**Step 3 — turn $Q_\pi$ into $A_\pi$.** Since $\sum_a \pi(a;s)Q_\pi(s,a) = V_\pi(s)$ and $\sum_a \pi'(a;s) = 1$:

$$\sum_a \pi'(a;s)\,Q_\pi(s,a) - V_\pi(s) = \sum_a \pi'(a;s)\big(Q_\pi(s,a) - V_\pi(s)\big) = \mathbb{E}_{a\sim\pi'(\cdot;s)}\big[A_\pi(s,a)\big]$$

$$\left.\frac{\partial \eta_\mu}{\partial \alpha}\right|_{\alpha=0} = \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d_{\pi,\mu}}\Big[\mathbb{E}_{a\sim\pi'(\cdot;s)}\big[A_\pi(s,a)\big]\Big] = \frac{1}{1-\gamma}\,\mathbb{A}_{\pi,\mu}(\pi') \qquad \square$$

The factor $\frac{1}{1-\gamma}$ comes from our normalization: $d_{\pi,\mu}$ sums to 1 and $Q_\pi$ carries a $(1-\gamma)$.

### $\alpha$ and $\theta$ play the same role

| | **CPI: parameter $\alpha \in [0,1]$** | **TRPO: parameter $\theta \in \mathbb{R}^m$** |
|---|---|---|
| first order | $\eta_\mu(\pi_\alpha) = \eta_\mu(\pi) + \frac{\alpha}{1-\gamma}\mathbb{A}_{\pi,\mu}(\pi') + O(\alpha^2)$ | $\eta(\pi_\theta) = L_{\pi_{\theta_0}}(\pi_\theta) + o(\lVert\theta - \theta_0\rVert)$ |
| source | this proof | TRPO Eq. 4 |
| step size | $\alpha$ itself | $\alpha = D_{\mathrm{TV}}^{\max}(\pi_{\mathrm{old}}, \pi_{\mathrm{new}})$ (Thm. 1) |

**Key idea**

The proof never used that $\alpha$ is a mixing weight — only that it parameterizes the policy. In a mixture, $\alpha$ is **both** the parameter and the distance (two mixtures differing by $\alpha$ disagree w.p. at most $\alpha$). For a general $\pi_\theta$ these are different things — which is why TRPO needs $D_{\mathrm{TV}}$, then KL.

## B Comparing with Bertsekas & Tsitsiklis

Our comparison (not in the paper), in five steps. Notation: $\bar{A}$ as in Lemma 6.1, $\Delta\eta = \eta_\mu(\pi') - \eta_\mu(\pi)$, and $(T_{\pi'}V)(s) = \mathbb{E}_{a\sim\pi'}\big[(1-\gamma)\mathcal{R}(s,a) + \gamma\,\mathbb{E}_{s'}V(s')\big]$, so that $\bar{A} = T_{\pi'}V_\pi - V_\pi$.

### B.1 The B&T bound: a worst case at every state

Given $\varepsilon = \max_s |\tilde{V}(s) - V_\pi(s)|$ and $\pi'$ greedy for $\tilde{V}$ (i.e. $T_{\pi'}\tilde{V} \geq T_\pi \tilde{V}$):

$$\begin{aligned} T_{\pi'}V_\pi &\geq T_{\pi'}\tilde{V} - \gamma\varepsilon && V_\pi \text{ vs. } \tilde{V} \text{ at the next state: error} \leq \varepsilon, \text{ times } \gamma \\ &\geq T_\pi \tilde{V} - \gamma\varepsilon && \pi' \text{ is greedy for } \tilde{V} \\ &\geq T_\pi V_\pi - 2\gamma\varepsilon \;=\; V_\pi - 2\gamma\varepsilon && \text{the same error again} \end{aligned}$$

$$\bar{A}(s) \;\geq\; -2\gamma\varepsilon \;\text{ at } \textbf{every } s \quad\Longrightarrow\quad \Delta\eta = \frac{1}{1-\gamma}\,\mathbb{E}_{d_{\pi',\mu}}[\bar{A}] \;\geq\; -\frac{2\gamma\varepsilon}{1-\gamma}$$

**Only a floor**

$\varepsilon$ is **how wrong our values are**. It only limits how **negative** $\bar{A}$ can be; nothing positive is known, since $\pi'$ is greedy for the *wrong* values. ($\varepsilon = 0$: $\bar{A} \geq 0$ everywhere — exact policy improvement.) Because the floor holds at every state, we never need $d_{\pi',\mu}$.

### B.2 Theorem 4.1 at $\alpha = 1$

Split the new states into old states + shift:

$$(1-\gamma)\Delta\eta = \sum_s d_{\pi',\mu}(s)\,\bar{A}(s) = \underbrace{\sum_s d_{\pi,\mu}(s)\,\bar{A}(s)}_{=\mathbb{A}\ \text{(measurable)}} + \underbrace{\sum_s \big(d_{\pi',\mu} - d_{\pi,\mu}\big)(s)\,\bar{A}(s)}_{\text{shift}}$$

1. Shift $\geq -\|d_{\pi',\mu} - d_{\pi,\mu}\|_1 \cdot \varepsilon$, with $\varepsilon = \max_s |\bar{A}(s)| = \max_s |\mathbb{E}_{a\sim\pi'} A_\pi(s,a)|$. It needs $|\cdot|$: the shift can also move mass *away* from states where $\bar{A} > 0$.
2. Same start $\mu \Rightarrow$ only $t \geq 1$ can differ: $\|d_{\pi',\mu} - d_{\pi,\mu}\|_1 \leq (1-\gamma)\sum_{t\geq 1}\gamma^t \cdot 2 = 2\gamma$.

$$\Delta\eta \;\geq\; \frac{\mathbb{A}}{1-\gamma} - \frac{2\gamma\varepsilon}{1-\gamma} \qquad = \text{Thm 4.1 at } \alpha = 1$$

The new ingredient is $\mathbb{A}$: B&T never measure any average of the true $\bar{A}$. The $2\gamma$ is the same number as in B&T, from a different place: there, two estimation errors one step ahead; here, two state distributions that agree at the start.

### B.3 Why the mixture helps

Same split for $\pi_{\text{new}} = (1-\alpha)\pi + \alpha\pi'$:

1. **Per state**: $\bar{A}_{\text{new}}(s) = \alpha\bar{A}(s) \Rightarrow$ gain $\alpha\mathbb{A}$, $\max_s |\bar{A}_{\text{new}}| = \alpha\varepsilon$.
2. **Shift**: $\pi_{\text{new}}$ acts like $\pi$ w.p. $1-\alpha$ each step, so $\Pr(\text{deviated by } t) \leq 1 - (1-\alpha)^t$, and
$$\|d_{\pi_{\text{new}},\mu} - d_{\pi,\mu}\|_1 \leq (1-\gamma)\sum_t \gamma^t\, 2\big(1-(1-\alpha)^t\big) = \tfrac{2\alpha\gamma}{1-\gamma(1-\alpha)}$$
3. **Multiply**: $(1-\gamma)\Delta\eta \geq \alpha\mathbb{A} - \alpha\varepsilon \cdot \frac{2\alpha\gamma}{1-\gamma(1-\alpha)}$ $\quad =$ Thm 4.1.

**Key idea**

Both factors of the penalty carry an $\alpha \Rightarrow \alpha^2$. Small $\alpha$: the gain wins. $\alpha = 1$: since $\mathbb{A} \leq \varepsilon$ we would need $\mathbb{A} > 2\gamma\varepsilon$ — impossible for $\gamma \geq \frac{1}{2}$. The positive term alone does not justify the full greedy step; **the small step does**.

### B.4 Same penalty, different $\varepsilon$

| | B&T | K&L, Thm 4.1 | TRPO, Thm 1 |
|---|---|---|---|
| $\varepsilon =$ | $\max_s \lvert\tilde{V}(s) - V_\pi(s)\rvert$ | $\max_s \lvert\mathbb{E}_{a\sim\pi'} A_\pi(s,a)\rvert$ | $\max_{s,a} \lvert A_\pi(s,a)\rvert$ |
| depends on | our estimate $\tilde{V}$ | the pair $\pi, \pi'$ | $\pi$ only |
| measures | how wrong our values are | largest one-step gain/loss of $\pi'$ | scale of the advantages |
| step size | fixed: full greedy step | $\alpha$ (mixture weight) | $\alpha = D_{\mathrm{TV}}^{\max}(\pi, \tilde{\pi})$ |

- **Common**: the price of not knowing $\bar{A}$ on the states the new policy visits.
- **Why it changed**: in B&T, $\pi'$ is fixed by $\tilde{V}$, so only the estimate can go wrong. K&L ask *how far may we step?* — so $\varepsilon$ measures the step, not the estimate. It cannot be estimated either (a max over all states), so Cor. 4.2 replaces it by $R$.

### B.5 Where the error went, and what TRPO does

**Where did B&T's approximation error go?** Not into Thm 4.1's $\varepsilon$, but into

1. the estimate of $\mathbb{A}$ from $\mu$-restarts (sampling error, Hoeffding);
2. the accuracy of the greedy chooser $G_\varepsilon$.

Both are **averages** under $d_{\pi,\mu}$ — much weaker than B&T's $\max_s$. (The $\varepsilon$ of $G_\varepsilon$ and Thm 4.4 is the chooser's accuracy, not the $\varepsilon$ of Thm 4.1.)

**TRPO splits K&L's** $\varepsilon$ into a step and a scale (TRPO App. A, Lemma 2: couple $a \sim \pi$, $\tilde{a} \sim \tilde{\pi}$ to disagree w.p. at most $\alpha$; use $\mathbb{E}_{a\sim\pi} A_\pi = 0$):

$$|\bar{A}(s)| \;\le\; 2 \underbrace{D_{\mathrm{TV}}\big(\pi(\cdot|s), \tilde{\pi}(\cdot|s)\big)}_{\text{policy change} \le \alpha} \cdot \underbrace{\max_{s,a} |A_\pi(s,a)|}_{\text{advantage scale}}$$

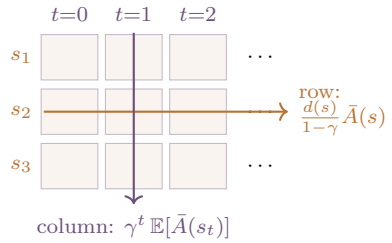
**Key idea**

The step becomes a **distance between policies** that the algorithm controls ⇒ trust region. Plugging this into the K&L argument gives the $\frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2$ penalty, with $4 = 2 \cdot 2$.

## C From $\mathbb{E}_{s\sim d}$ to a sum over time

The second equality of Lemma 6.1, in four steps:

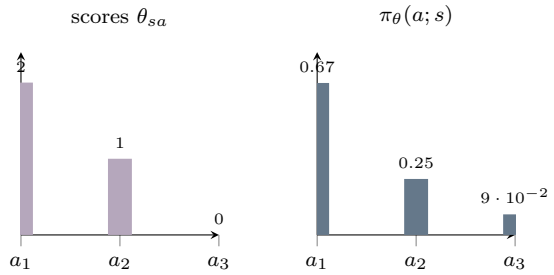
$$\begin{aligned}
\frac{1}{1-\gamma}\mathbb{E}_{s\sim d_{\pi',\mu}}[\bar{A}(s)] &= \frac{1}{1-\gamma}\textstyle\sum_s d_{\pi',\mu}(s)\,\bar{A}(s) && \text{1. expectation = weighted sum}\\
&= \textstyle\sum_s \Big[\sum_t \gamma^t \Pr(s_t{=}s)\Big]\bar{A}(s) && \text{2. definition of } d\text{; } (1-\gamma) \text{ cancels}\\
&= \textstyle\sum_t \gamma^t \Big[\sum_s \Pr(s_t{=}s)\,\bar{A}(s)\Big] && \text{3. swap the sums}\\
&= \textstyle\sum_t \gamma^t\, \mathbb{E}\big[\bar{A}(s_t)\big] = \mathbb{E}\Big[\sum_t \gamma^t \bar{A}(s_t)\Big] && \text{4. definition of } \mathbb{E}\text{; linearity}
\end{aligned}$$




- **Cell** $(s,t)$: $\gamma^t \Pr(s_t{=}s)\,\bar{A}(s)$.
- Sum by **rows** (step 2) or by **columns** (step 3): same total.
- The swap is allowed: $|\text{cell}| \le \gamma^t \Pr(s_t{=}s)R$, and these sum to $R/(1-\gamma) < \infty$ (absolute convergence).

$\mathbb{E}[\bar{A}(s_t)]$ is over $s_t \sim p_t$, $p_t(s) = \Pr(s_t{=}s \mid s_0 \sim \mu, \pi')$ — the state at time $t$, **not discounted**; one distribution per $t$. $\mathbb{E}[\sum_t \cdots]$ is over trajectories $s_0 \sim \mu$, $a_t \sim \pi'$, $s_{t+1} \sim P$. The discount sits only in the weights $\gamma^t$.

# D The Gibbs (softmax) policy




One parameter per state–action pair ("table lookup"):

$$\pi_\theta(a;s) = \frac{e^{\theta_{sa}}}{\sum_b e^{\theta_{sb}}}$$

One state, three actions. Every action keeps **some** probability.

1. **Why "Gibbs"**: the Boltzmann distribution of physics, $p \propto e^{-E/T}$, with $\theta = -E/T$. A temperature $(\theta/T)$: $T \to 0$ gives argmax, $T \to \infty$ uniform.
2. **Always stochastic**: $\pi_\theta(a;s) > 0$; deterministic only as $\theta \to \pm\infty$.
3. **Rich**: any policy with full support, in every state independently. So the trouble in Example 2 comes from the *gradient*, not from the class.
4. **Not unique**: adding a constant to all $\theta_{sa}$ of one state leaves $\pi$ unchanged.
5. **Simple score** (derivation below): $\partial_{\theta_{sb}} \log \pi_\theta(a;s) = \mathbf{1}[a=b] - \pi_\theta(b;s)$.

## Deriving the score

One state, so drop $s$ ($\theta_a = \theta_{sa}$):

$$\pi(a) = \frac{e^{\theta_a}}{\sum_c e^{\theta_c}} \quad\Longrightarrow\quad \log\pi(a) = \theta_a - \log\textstyle\sum_c e^{\theta_c} \quad\Longrightarrow\quad \partial_{\theta_b}\log\pi(a) = \underbrace{\partial_{\theta_b}\theta_a}_{\mathbf{1}[a=b]} - \underbrace{\frac{e^{\theta_b}}{\sum_c e^{\theta_c}}}_{\pi(b)}$$

- **Vector form**: $\nabla_{\theta_s} \log\pi(a;s) = e_a - \pi(\cdot\,;s)$ — "the action taken minus the average action". Other states get derivative 0.
- **Check**: $\mathbb{E}_{a\sim\pi}\big[\mathbf{1}[a=b] - \pi(b)\big] = 0$ — the identity $\mathbb{E}[\nabla \log\pi] = 0$ that makes baselines unbiased.
- **In REINFORCE**: taking $a_t$ with $\hat{Q}_t > 0$ raises $\theta_{sa_t}$ by $\alpha\hat{Q}_t(1-\pi(a_t;s))$ and lowers each other $\theta_{sb}$ by $\alpha\hat{Q}_t\pi(b;s)$. Only visited states get gradient — why a rarely visited state $j$ learns slowly.
- Two actions: $\pi(a_1) = \sigma(\theta_1 - \theta_2)$ and $(\log\sigma)' = 1 - \sigma$.

# E Why the log-derivative trick is unbiased

$\theta$ sits in the **sampling distribution**, not in $R(\tau)$, so averaging $\nabla_\theta R$ gives nothing. The trajectory density:

$$p_\theta(\tau) = D(s_0) \prod_t \pi_\theta(a_t; s_t)\, P(s_{t+1}; s_t, a_t)$$

1. Use $\nabla p = p\, \nabla \log p$ wherever $p > 0$:

$$\nabla_\theta\, \eta_D = \sum_\tau \nabla_\theta\, p_\theta(\tau)\, R(\tau) = \mathbb{E}_{\tau\sim\pi_\theta}\big[R(\tau)\, \nabla_\theta \log p_\theta(\tau)\big]$$

2. The log turns $\prod$ into $\sum$, and the grey terms have no $\theta$:

$$\nabla_\theta \log p_\theta(\tau) = \textstyle\sum_t \nabla_\theta \log \pi_\theta(a_t; s_t)$$

**Key idea**

$\hat{g} = R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t; s_t)$, $\tau \sim \pi_\theta \Rightarrow \mathbb{E}[\hat{g}] = \nabla_\theta \eta_D$ **exactly**, and no need to know $P$.

**Unbiased is not low variance.** Two standard changes keep it unbiased and reduce the noise:

1. Weight each $\nabla \log \pi_\theta(a_t; s_t)$ only by rewards **after** $t$ (an action cannot affect earlier rewards) — the $\hat{Q}_t$ of REINFORCE.
2. Subtract a **baseline** $b(s_t)$, since $\mathbb{E}_{a\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(a; s)] = \sum_a \nabla_\theta \pi_\theta(a; s) = \nabla_\theta 1 = 0$. With $b = V_\pi$ the weight becomes the advantage — the road to GAE.

Williams (1992); also called the score-function or likelihood-ratio estimator. $R(\tau) = (1-\gamma)\sum_t \gamma^t r_t$.

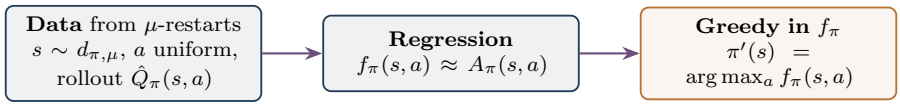
# F CPI through the SGD lens

Take the table $\pi(a; s)$ itself as the parameter: CPI is a **first-order method** on $\eta_\mu$.

| | SGD (non-convex, smooth) | CPI |
|---|---|---|
| direction | $\hat{g} \approx \nabla J(w)$ | greedy $\pi' = G_\varepsilon(\pi, \mu)$: best linearized direction |
| step | $w \leftarrow w + a_k \hat{g}$ | $\pi \leftarrow (1-\alpha)\pi + \alpha\pi'$ (Frank–Wolfe step) |
| step size | $\sum_k a_k = \infty$, $\sum_k a_k^2 < \infty$ | $\alpha \propto \hat{\mathbb{A}}$: slope over a smoothness bound |
| noise | averages out over many steps | each step accurate w.h.p. (Hoeffding) |
| stationary | $\|\nabla J(w_R)\| \le \epsilon$, random iterate | $\mathrm{OPT}(\mathbb{A}_{\pi,\mu}) < 2\varepsilon$, last iterate |
| cost | $O(\sigma^2/\epsilon^4)$ samples | $72R^2/\varepsilon^2$ updates $\times$ $\tilde{O}(R^2/\varepsilon^2)$ rollouts |
| global? | $\times$ a stationary point can be bad | ✓ Thm. 6.2, up to the mismatch factor |

1. **CPI is Frank–Wolfe on the table $\pi(a; s)$.** $\partial\eta_\mu/\partial\pi(a; s) = d_{\pi,\mu}(s) Q_\pi(s, a)/(1-\gamma)$, so the slope toward $\pi'$ is $\langle \nabla\eta_\mu, \pi' - \pi\rangle = \mathbb{A}_{\pi,\mu}(\pi')/(1-\gamma)$. $G_\varepsilon$ maximizes the linearization; per state the maximum is the greedy vertex $\arg\max_a A_\pi(s, a)$. $\mathrm{OPT}(\mathbb{A})/(1-\gamma)$ is the Frank–Wolfe gap. Caveat: $\eta_\mu$ is not concave in $\pi$. (Vieillard, Pietquin & Geist 2019; Scherrer & Geist 2014; Lacoste-Julien 2016.)
2. **Gibbs parameterization.** $\partial\eta_\mu/\partial\theta_{sa} = d_{\pi,\mu}(s)\pi(a; s)A_\pi(s, a)/(1-\gamma)$: the same advantage, but times $\pi(a; s)$, so good unlikely actions and unvisited states barely move (Ex. 1–2). The natural gradient (Kakade 2001) drops both factors: $\pi_{\text{new}} \propto \pi\, e^{\beta A_\pi}$, a soft move toward greedy. CPI's mixture is the hard version.
3. **Link to SGD.** SGD lets unbiased noise cancel over many steps; CPI makes each step accurate to $\pm\varepsilon/3$ w.h.p. and steps only if $\hat{\mathbb{A}} \ge 2\varepsilon/3$: monotone improvement, a guarantee for the last iterate. Counting (gain $\ge \varepsilon^2/72R$, $\eta_\mu \le R$) is the descent-lemma argument. Thm. 6.2 is what later theory calls **gradient domination** (Agarwal, Kakade, Lee & Mahajan 2021).

# G Building $G_\varepsilon$ by regression on advantages




**Definition: required accuracy (K&L Sec. 7.1)**

$$L(f_\pi) = \underbrace{\mathbb{E}_{s\sim d_{\pi,\mu}}}_{\text{average over states}} \underbrace{\max_a}_{\text{worst action}} \left|A_\pi(s,a) - f_\pi(s,a)\right| \le \frac{\varepsilon}{2}$$

**Why it suffices.** With $e(s) = \max_a |A_\pi - f_\pi|$, $\hat{a} = \arg\max_a f_\pi$, $a^* = \arg\max_a A_\pi$:

$$A_\pi(s,\hat{a}) \ge f_\pi(s,\hat{a}) - e(s) \ge f_\pi(s,a^*) - e(s) \ge A_\pi(s,a^*) - 2e(s)$$
$$\mathbb{E}_{s\sim d_{\pi,\mu}}: \quad \mathbb{A}_{\pi,\mu}(\pi') \ge \mathrm{OPT}(\mathbb{A}_{\pi,\mu}) - 2L \ge \mathrm{OPT}(\mathbb{A}_{\pi,\mu}) - \varepsilon$$

**Key idea**

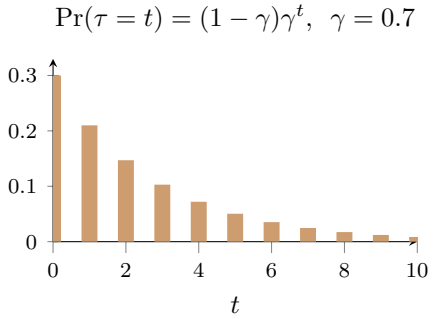
An **average** ($\ell_1$) error over $d_{\pi,\mu}$, not the $\max_s$ ($\ell_\infty$) error of greedy DP (Gap 1): rarely visited states may be fit badly.

- **Fitting $Q$ is enough**: $A_\pi = Q_\pi - V_\pi(s)$ shifts each state by a constant — same error, same argmax.
- **Link to usual losses**: $\max_a |e| \le n_a \mathbb{E}_{a\sim\mathrm{unif}}|e| \le n_a\sqrt{\mathbb{E}e^2}$, so an MSE below $(\varepsilon/(2n_a))^2$ suffices (pessimistic).
- The paper does not bound the samples this regression needs, so the total cost of CPI is open. Descendants: API with classifiers (Lagoudakis & Parr 2003).

# H Stopping with probability $1-\gamma$ samples $d_{\pi,\mu}$

Run $\pi$ from $s_0 \sim \mu$; after each step stop w.p. $1-\gamma$, independently of the trajectory. Return $s_\tau$.

1. $\Pr(\tau = t) = \gamma^t(1-\gamma)$ ($t$ continues, then one stop).
2. $\Pr(s_\tau = s) = \sum_t \Pr(\tau = t)\Pr(s_t = s)$ (needs independence).
3. $= (1-\gamma)\sum_t \gamma^t \Pr(s_t = s \mid \pi, \mu) = d_{\pi,\mu}(s)$.




**Key idea**

The discount weights $(1-\gamma)\gamma^t$ are a **geometric distribution** over $t$. $\gamma$ = probability of surviving one more step; $d_{\pi,\mu}$ = the distribution of the state where an episode of random length ends. The $(1-\gamma)$ in $d_{\pi,\mu}$ is what makes $\sum_t \Pr(\tau = t) = 1$.

Cost of one sample: $\mathbb{E}[\tau] = \gamma/(1-\gamma)$, about 99 steps at $\gamma = 0.99$.

# I Hoeffding's inequality and the estimate of $\mathbb{A}$

**Hoeffding (1963)**

If $X_1, \dots, X_k$ are **independent** and $X_i \in [a, b]$ a.s., then for $\bar{X} = \frac{1}{k}\sum_i X_i$:

$$\Pr\big(|\bar{X} - \mathbb{E}\bar{X}| > \Delta\big) \;\le\; 2\exp\Big(-\frac{2k\Delta^2}{(b-a)^2}\Big)$$

**Conditions for the $x_i$ of CPI:**

1. **Independent**: each $x_i$ comes from a fresh $\mu$-restart.
2. **Bounded**: $\hat{Q}_i \in [0, R]$, $|\pi' - \pi| \le 1 \Rightarrow x_i \in [-n_a R, n_a R]$, $b - a = 2n_a R$.
3. **Right mean**: $\mathbb{E}x_i = \mathbb{A}$ (uniform $a$, weight $n_a$); the bias of a finite rollout is kept below $\varepsilon/6$.

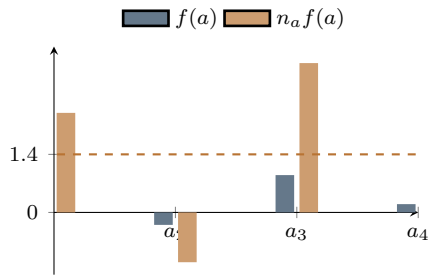
$$\Pr\big(|\hat{\mathbb{A}} - \mathbb{A}| > \Delta\big) \le 2e^{-k\Delta^2/(2n_a^2R^2)} \quad\Longrightarrow\quad k \ge \frac{2n_a^2R^2}{\Delta^2}\log\frac{2}{\delta} \;\text{ gives error } \le \Delta \text{ w.p. } \ge 1-\delta$$

No identical distribution is needed, and no variance enters — the bound is loose when the $x_i$ rarely reach the ends of the range. The range $2n_aR$ is what costs: the importance weight $n_a$ widens it, hence $k \propto n_a^2$. In CPI, $\Delta = \varepsilon/3$ and $\delta$ is split over at most $72R^2/\varepsilon^2$ updates (union bound), which gives the $O\big(\frac{R^2}{\varepsilon^2}\log\frac{R^2}{\delta\varepsilon^2}\big)$ restarts per update (the paper drops $n_a$).

# J Importance sampling in CPI, and why uniform actions

The estimate of $\mathbb{A}$ needs a **sum** over actions, but each rollout tries **one** action.

1. **Sample from $q$, reweight by $w = p/q$** (needs $q > 0$ where $p > 0$):
$$\mathbb{E}_{a\sim p}[f] = \textstyle\sum_a q\,\frac{p}{q}\,f = \mathbb{E}_{a\sim q}\big[w\,f\big]$$
2. **A plain sum is the case $p \equiv 1$**; CPI takes $q$ uniform:
$$\textstyle\sum_a f(a) = \mathbb{E}_{a\sim q}\Big[\frac{f(a)}{q(a)}\Big] = \mathbb{E}_{a\sim\text{unif}}\big[n_a f(a)\big]$$
3. **Unbiased, but the weight stretches the range**: $|f| \le R \Rightarrow |n_a f| \le n_a R \Rightarrow n_a^2$ in Hoeffding.



Four actions, $\sum_a f = 1.4$; the orange values average to 1.4.

### Why uniform? Two ways to sample the same target

At a state $s \sim d_{\pi,\mu}$:

$$\textstyle\sum_a \big(\pi' - \pi\big)(a;s)\,Q_\pi(s,a) \;=\; \underbrace{\mathbb{E}_{a\sim\text{unif}}\big[n_a(\pi' - \pi)(a;s)\,Q_\pi(s,a)\big]}_{\text{importance sampling, } p\equiv 1} \;=\; \underbrace{\mathbb{E}_{a\sim\pi'}\big[Q_\pi(s,a)\big] - V_\pi(s)}_{\text{sample from } \pi'}$$

**CPI: $a$ uniform, one rollout of $\pi$**

$x = n_a(\pi' - \pi)(a;s)\,\hat{Q}(s,a) \in [-n_aR, n_aR]$
- \+ same data as $G_\varepsilon$ (needs $\hat{Q}$ for *every* $a$)
- \+ one batch scores *any* candidate $\pi'$
- − factor $n_a^2$ in the sample count

**Alternative: $a \sim \pi'$, rollouts of $\pi$**

$x = \hat{Q}(s,a) - \hat{V}(s) \in [-R, R]$
- \+ no $n_a$; still only rollouts of $\pi$ (Q2 holds)
- − needs $\pi'$ *before* sampling
- − a second estimate per state ($\hat{V}$)

**Not plain IS with $q = \pi'$**

The weight would be $1/\pi'(a)$, but a greedy $\pi'$ is often deterministic, so $\pi'(a) = 0$ where $\pi(a)Q_\pi(s,a) \neq 0$ and the identity fails. The subtraction of $V_\pi$ in the second form is the same baseline idea that later appears as the advantage in TRPO/PPO.

## K TRPO with continuous states and actions

TRPO footnote 1: "replace the sums with integrals". **True** — no proof uses finiteness. But "integrals" hides some conditions.

**Why it carries over**

- **Performance difference**: telescoping, any space.
- **Coupling**: mass $1 - D_{\mathrm{TV}}$ on $\{a = \tilde{a}\}$ — also for densities (maximal coupling on standard Borel spaces).
- **Perturbation** (App. B): matrices → Markov operators, same norms.

**What "integrals" hides**

1. **Bounded** $\epsilon = \sup_{s,a} |A_\pi(s,a)| < \infty$.
2. **max → sup** over **all** states, not just visited ones.
3. **Stochastic** policies: deterministic ⇒ $D_{\mathrm{KL}} = \infty$.
4. **Measurable** kernels; $\rho_\pi$ is a measure.

Condition 1 is often **vacuous** in continuous control: Gaussian actions with reward $-\|u\|^2$ (their Swimmer: $r = v_x - 10^{-5}\|u\|^2$) give $\epsilon = \infty$. The practical algorithm drops the bound anyway (mean KL, fixed $\delta$). For a Gaussian with fixed $\sigma$: $D_{\mathrm{KL}}(s) = \|\mu_{\mathrm{new}}(s) - \mu_{\mathrm{old}}(s)\|^2 / 2\sigma^2$.

## L One problem, several known algorithms

| Objective | Constraint / penalty | Algorithm |
|---|---|---|
| linear in $\theta$ | $\frac{1}{2}\|\theta - \theta_{\mathrm{old}}\|^2 \leq \delta$ | vanilla policy gradient |
| linear in $\theta$ | quadratic KL, fixed step $\frac{1}{\lambda}$ | natural policy gradient |
| full $L_{\pi_{\mathrm{old}}}(\pi)$ | none | policy iteration |
| $L_{\theta_{\mathrm{old}}}(\theta)$ | $\bar{D}_{\mathrm{KL}} \leq \delta$, enforced | **TRPO** |

- **NPG vs. TRPO**: the same direction $A^{-1}g$; TRPO **enforces** the KL constraint at each step — this matters on larger problems.
- **Related**: REPS constrains the joint $p(s,a)$; TRPO constrains only $\pi(a \mid s)$.

**Source:** TRPO Sec. 7.